Interaktív feladatok
Kitalálós feladatok

Szerző: Horváth Gyula
Előadó: Leitereg András

2026. június 30.

Az interaktív feladatok főbb típusai

1. Kétszemélyes játékok

1.1 Pontgyűjtő

A két játékos felváltva lép, minden lépésben meghatározott mennyiségű pontot kap. Az győz, aki több pontot szerez. Az ellenfél lehet csalfa, tehát úgy lép, hogy a másik a lehető legkevesebb összpontot szerezze. A kezdő játékos játékát kell megvalósítani. .

1.2 Kitalálós feladatok

A kérdező játékát kell megvalósítani, aki kérdéseket (korlátozottan) tehet fel, hogy meghatározza a kitalálandót. A játékos akkor nyer, ha meg tudja határozni a kitalálandót.

1.3 Az veszít, aki nem tud lépni

A két játékos meghatározott szabályok szerint felváltva lép. Az nyer, aki utoljára tud lépni. A kezdő játékos stratégiáját kell megvalósítani.
Kétszemélyes játékok nem interaktív formában is előfordulhatnak IOI-n, például úgy, hogy az összes nyerő kezdőállást kell kiszámítani. Vagy adott kezdeti játékállásokról kell eldönteni, hogy azok nyerők-e?

2. Kódolós feladatok

Olyan programot kell készíteni, amelyet kétszer futtatnak. Az első alkalommal adott bemenetet kell kódját kell kimenetként megadni, a második alkalommal a program csak az első futtatás kimenetét (esetleg pl. méret információt) kap meg a program bemenetként, ebből kell dekódolnia.

3. Online feladatok

A probléma teljes bemenetét nem kapjuk meg egyszerre. Mielőtt további bemeneti adatok kapnánk, az addig megkapott adatokra kell számítás eredményét megadni

4. Kitalálós feladatok

A válasz meghatározásához korlátozott módon kérdethetünk. A megoldásra kapott pontszám függhet a feltett kérdések számától.

IOI-stílusu feladatok

Nem főprogramként kell megadni a feladat megoldását, hanem egy függvényként. A megoldás függvényét az értékelő greder hívja meg. A **greder** program végzi az input és output műveleteket, a megoldó függvény paraméterként kapja a bemenetet és a kiszámítandó kimenetet a függvény értékeként, vagy kimenő paraméterként adja át az értékelőnek.

```
1   #include <bits/stdc++.h>
2   #include "grader.h"
3   //saját adatok
4   //saját függvények
5   void Solution(Tbe Bemenet, Tki& Kimenet){
6     //a feladat megoldásának algoritmusa
7   }
```

Online feladatok

Feladat:K-adik legkisebb

Egész számoknak egy sorozatát kell feldolgozni. Egymás után kapjuk a sorozat következő elemét, ki kell számítani, hogy az eddig beérkezett adatok közül melyik a nagyság szerint K-adik különböző elem!

A `getK` és a `data` függvényt kell megvalósítanod!

Könyvtári műveletek

`void getK(int K)`: az értékelő egyszer hívja a végrehajtás elején, a K értékét adja át paraméterként

`int data(int a)`: az értékelő többször hívja, minden hívásnak az addig `data` művelettel megkapott számot közül a K-adik különbözőt kell visszaadnia. Ha nem érkezett addig K különböző, akkor a legnagyobbat adja vissza.

A letölthető minta `kkadik.zip` tartalmaz egy minta értékelőt

(`grader.cpp`), amely nem ellenőrzi a helyességet, de átalakíthatod úgy, hogy ellenőrizze.
A minta grader a standard bemenet első sorából olvassa be az N és K értéket, ahol N az adatsor elemszáma. A második sor tartalmazza az $A = (a_1, \ldots, a_N)$ adatsort. A `data` függvény i-edik hívása esetén a peremétere a_i lesz.
N értéke legfeljebb 10 000 lehet.

Megoldás

Minden `data(a)` hívás tegye be az a adatot az S halmazba. Ha ezzel S elemszáma $K+1$ lesz, akkor töröljük a legnagyobbat. Adjuk vissza S legnagyobb elemét.

```
1  #include <bits/stdc++.h>
2  #include "grader.h"
3  using namespace std;
4  int k;
5  set <int> S;
6  void getK(int K){k=K;}
7  int data(int a){
8    S.insert(a);
9    if(S.size()>k){
10     auto it=S.end();
11     it=prev(it);
12     S.erase(it);
13   }
14   auto it=prev(S.end());
15   return *it;
16 }
```

Az online feladatok hatékony megoldása gyakran speciális adatszerkezet alkalmazását igényli. Láttuk a Fapac adatszerkezet ismertetésénél ilyen alkalmazást. Van azonban közvetlen könyvtári támogatás adatszerkezet (bináris keresőfa) kibővítésére az un. Policy Based Data Structure könyvtári modulokkal. Erre példa az online sorrendi statisztika megvalósítása.

Feladat: Sorrendi statisztika

A megvalósítandó műveletek:

`void adat(int a)`: Vegyük közé a statisztikában szereplők közé az *a* adatot.

`int hanyadik(int a)`: Adja meg, hogy a statisztikában szereplő különböző adatok között nagyság szerint hányadik az *a* adat. Ha nincs ilyen adat, akkor a -1 értéket adja.

`int kadik(int k)`: Adja meg a statisztikában szereplő különböző adatok közül a nagyság szerinti *k*-adikat. Minden hívás előtt teljesül, hogy van legalább *k* különböző elem a statisztikában.

Megoldás

A standard rendezett halmaz, a `set` adattípus olyan kiterjesztését tudjuk alkalmazni, amely a megvalósító bináris keresőfa (piros-fekete) minden pontjában tárolja kiegészítő információként az adat sorrendbeli sorszámát (0-tól kezdve). A deklarációban meg kell adni azt a függvényt, amely aktualizálja a kiegészítő információt. A típusdeklaráció általános alakja: `tree<p1, p2, p3, p4, p5>`

A paraméterek jelentése

p1: az adatelemek típusa, itt `int`

p2: az alap adattípus, itt a set, ami a `null_type`

p3: a halmaz elemein a rendezési reláció, itt `less<int>`

p4: a keresőfa megvalósítása, itt piros-fekete fa, ami `rb_tree_tag`

p5: a kiegészítő információt aktualizáló módszer, itt `tree_order_statistics_node__update`, amit a `tree_policy.hpp` könyvtár tartalmaz

```cpp
#include <bits/stdc++.h>
using namespace std;

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree<int, null_type,less<int>,
  rb_tree_tag,tree_order_statistics_node_update>
  Sorrend;

void adat(int a){
  Sorrend.insert(a);
}
int hanyadik(int a){
  if(Sorrend.find(a)==Sorrend.end()) return -1;
  return Sorrend.order_of_key(a)+1;
}
int kadik(int k){
  if(Sorrend.size()<k) return -1;
  return *Sorrend.find_by_order(k-1);
}
```

Online feladat: Kiegyenlítés

Adott pozitív egész számoknak az $A = (a_1, \ldots, a_N)$ sorozata és egy K pozitív egész szám. Azt mondjuk, hogy az A sorozat K-kiegyenlített, ha bármely két elemének az eltérése legfeljebb K, azaz minden $1 \leq i, j \leq N$ indexpárra $|a_i - a_j| \leq K$ teljesül.
Minden A sorozat K-kiegyenlíthetővé tehető oly módon, hogy minden a_i eleméhez hozzáadunk egy alkalmas d_i egész számot. Egy ilyen kiegyenlítés költsége $\sum_{i=1}^{N} |d_i|$.
Az a feladat, hogy megadjuk az A sorozat K-kiegyenlítésének a minimális költségét.
A feladatot interaktív módon kell megoldani. A sorozat elemeit sorban beolvasva, meg kell adni az addig beolvasott elemeket tartalmazó sorozat K-kiegyenlítésének a minimális költségét.

Könyvtár

A következő könyvtári műveleteket kell használni.

`int getN()` : a program elején kell egyszer hívni, a sorozat N elemszámát adja.

`int getK()` : a K értékét adja.

`int Data()` : a sorozat következő elemét adja.

`Solution(long long m)` : minden `Data()` hívás után ezzel kell közölni az addig megkapott sorozat K-kiegyengetésének m minimális költségét.

A programod tartalmazza az `#include "grader.h"` importáló sort!

Gyakorlás

A letölthető sample.zip egy minta `grader` modult tartalmaz. Ha a megoldásod az `egyenget.cpp`, akkor add a projekthez a `grader.h`, `grader.cpp` és `egyenget.cpp` állományokat.

A program a standard bemenetről olvassa az adatokat. Az első sor az N és a K értékét tartalmazza, a második sor tartalmazza az A sorozat elemeit. A `Solution` függvény nem ellenőrzi a megoldás helyességét. Az in1.txt és in2.txt példa bemenetek harmadik sora a helyes megoldási értékeket tartalmazza.
Csak az `egyenget.cpp` állományt kell beadnod.

Megoldás

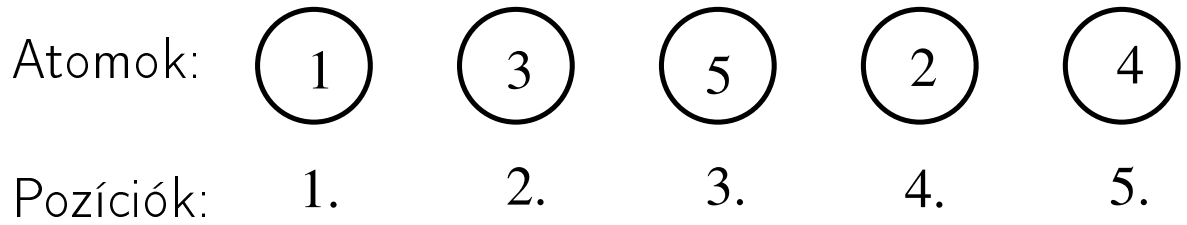
Vegyünk két prioritási sort, a *minA* minimumos, a *maxA* maximumos legyen. A *minA* minimuma mindig az eddig feldolgozott elemek minimuma legyen, az elvégzett kiegyenlítések utáni. Hasonlóan a *maxA* maximuma mindig az eddig feldolgozott elemek maximuma legyen, az elvégzett kiegyenlítések utáni.

```
1   #include <bits/stdc++.h>
2   using namespace std;
3   long long N, K, mego=0;
4   priority_queue<long long> maxA;
5   priority_queue<long long, vector<long long>,
6                  greater<long long> > minA;
7   int main(){
8     ios_base::sync_with_stdio(false);
9     long long adat;
10    N=getN, K=getK;;
11    for (int i=0; i<N; i++) {
12      adat=Data();
13      maxA.push(adat), minA.push(adat);
14      long long maxi=maxA.top(), mini=minA.top();
15      if (maxi>mini+K) {
16        mego+=maxi-(mini+K);
17        maxA.pop(), minA.pop();
18        maxA.push(mini+K), minA.push(maxi-K);
19      }
20      Solution(mego);
21    }
```

Kitalálós feladatok

Feladat: Atomlánc (CEOI'2001)

Kutatók egy speciális molekulát vizsgálnak. Tudják, hogy a molekula n különböző atomot tartalmaz, amelyek egy lineáris láncot alkotnak. A kutatóknak van egy olyan mérőműszerük, amellyel meg tudják mérni a molekula két adott atomjának a távolságát. A műszer által mért távolság a két atom pozíció különbségének abszolút értéke. Az elvégezhető mérések azonban korlátozottak, egyetlen atom sem szerepelhet négynél több mérésben, mert ez tönkretenné a molekulát.
Olyan programot kell írni, amely meghatározza a molekula szerkezetét, azaz minden atom pozícióját a molekulában!



1. ábra. Atomlánc példa

Könyvtár

A mérőműszer használatát a Meter könyvtár három művelete biztosítja:

`int Size()`: Egyszer kell hívni a program elején, az atomok n számát adja ($5 \leq n \leq 10000$).

`int Span(int a, int b)` Két atom sorszámát kell argumentumként megadni, a visszaadott érték a két atom távolsága.

`int Answer(int i, int x)` A program végén kell hívni, a kiszámított eredmény közléséhez. Két argumentuma van, i és x, ami azt jelenti, hogy a molekulában az i-edik pozíción a x sorszámú atom van. Minden i-re $(1 \leq i \leq n)$ pontosan egyszer kell hívni, tetszőleges sorrendben. A megoldás tükörkép erejéig egyértelmű, a két megoldás közül bármelyiket meg lehet adni.

Gyakorlás

A standard bemenet két sort tartalmazzon. Az első sorban legyen az atomok n száma. A második sor pontosan n különböző számot tartalmazzon egy-egy szóközzel elválasztva, az atomok sorszámait. Minden szám 1 és n közötti egész szám legyen.

5

1 3 5 2 4

Eljárás hívások, amelyek egy megoldását adják a példa bemenetnek:

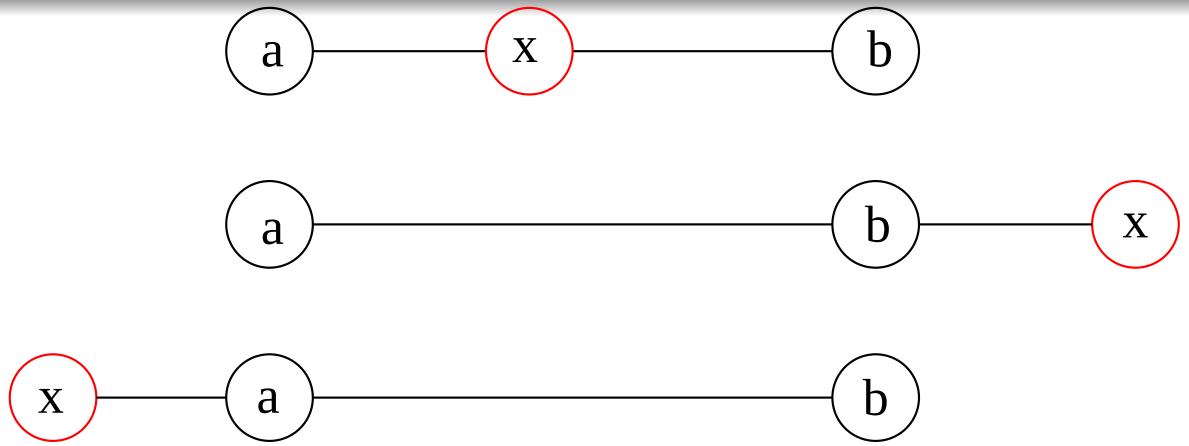
1. Size() 5 értéket ad
2. Span(1,3) 1-et ad
3. Span(2,5) 1-et ad
4. Span(3,5) 1-et ad
5. Span(1,4) 4-et ad
6. Answer(1,1) Answer(5,4) Answer(3,5) Answer(4,2) Answer(2,3)

Pontozás

Ha a közölt megoldás helyes és egyetlen atom sem szerepelt háromnál többször Span műveletben, akkor teljes pontszám (100%) jár. Ha a közölt megoldás helyes és egyetlen atom sem szerepelt négynél többször Span műveletben, de volt olyan atom amely négy Span-ben szerepelt, akkor fél pontszám (50%) jár. Minden más esetben 0 pont jár.

50%-os megoldás

Állítás Ha ismerjük két olyan atom, a és b relatív pozícióját (tehát távolságát is), hogy a eddig legfeljebb 3 kérdésben szerepelt, b pedig legfeljebb 2-ben, akkor tetszőleges x atom pozícióját ki tudjuk számítani, és ismét lesz két olyan atomunk, hogy az egyik 3, a másik legfeljebb 2 kérdésben szerepelt (nevezetesen b és x).
Bizonyítás Jelölje $Poz\,(a)$ az a atom már kiszámított relatív pozícióját. Először kérdezzük le $a-x$ és $b-x$ távolságát; $dax := Span\,(a, x)$; $dbx := Span\,(b, x)$ és legyen $dab = abs\,(Poz\,(a) - Poz\,(b))$. Ekkor az alábbi ábrán látható három lehetőség adódik, és látható, hogy mindegyik esetben kiszámítható x relatív pozíciója. Feltesszük, hogy $Poz\,(a) < Poz\,(b)$.

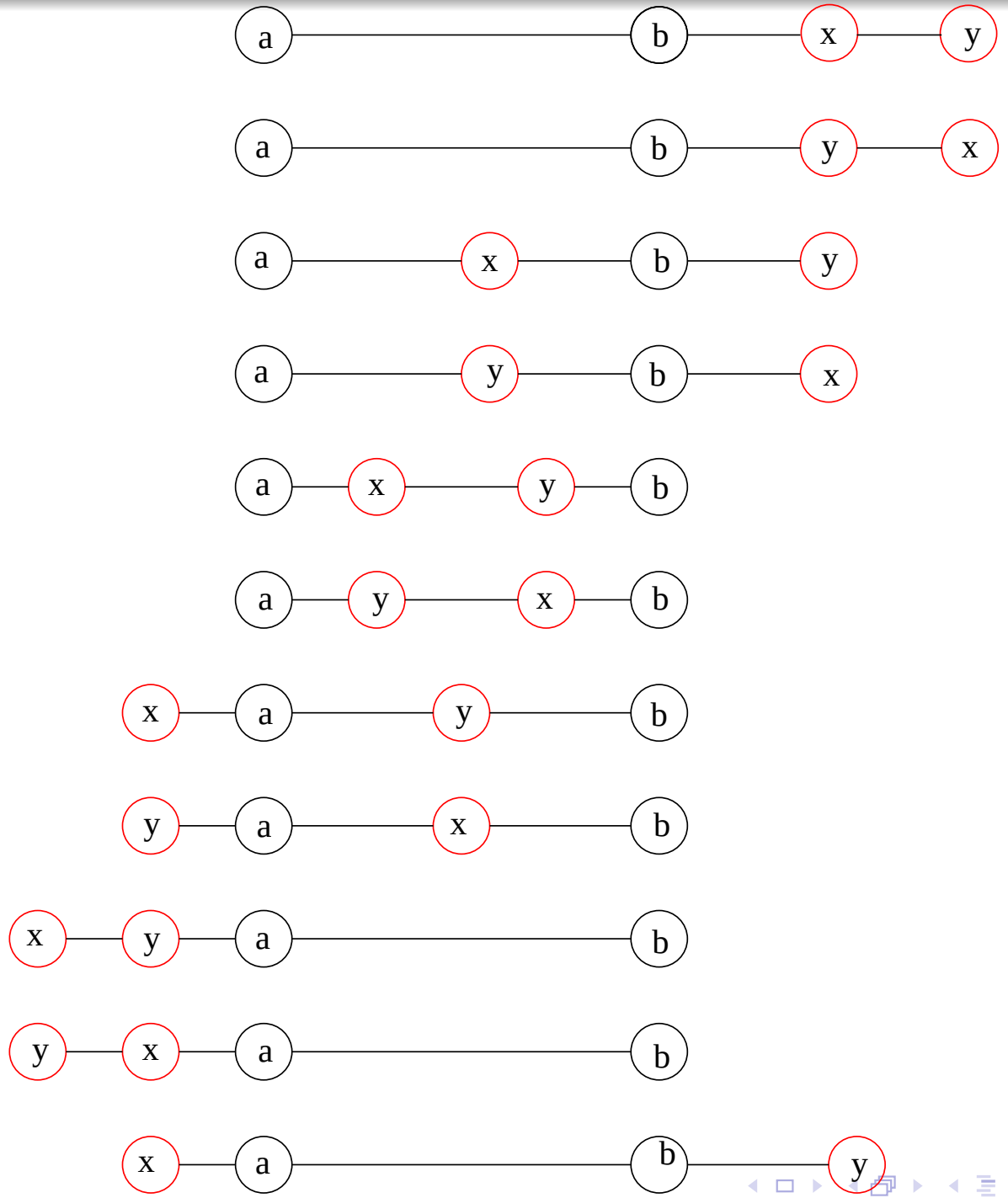


2. ábra. x pozíciójának meghatározása 2 kérdéssel.

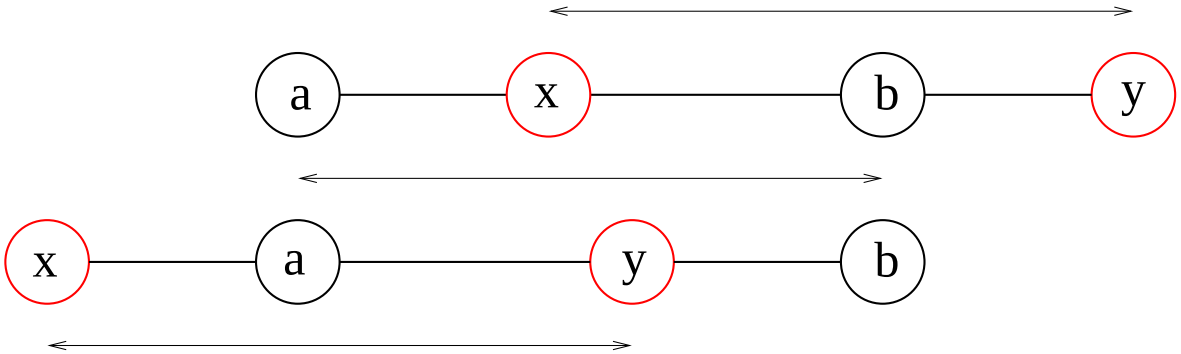
1. eset: $dax < dab$ és $dbx < dab$, ekkor $Poz(x) = Poz(a) + dax$
2. eset: $dax = dab + dbx$, ekkor $Poz(x) = Poz(b) + dbx$
3. eset: $dbx = dab + dax$, ekkor $Poz(x) = Poz(a) - dax$

Teljes (100%-os) megoldás

Tegyük fel, hogy ismerjük az a és a b atom relatív pozícióját, és mindkettő eddig legfeljebb 2 kérdésben szerepelt. Először kérdezzük le két atom, x és y távolságát, majd az $a-x$ és a $b-y$ távolságot. Ekkor meg tudjuk határozni x és y pozícióját, továbbá ismét lesz két atomunk, amelyek 2 kérdésben szerepeltek.


a b x y
a b y x
a x b y
a y b x
a x y b
a y x b
x a y b
y a x b
x y a b
y x a b
x a b y

3. ábra. x és y lehetséges elhelyezkedése $a - b$-hez képest.



4. ábra. **Vigyázat!**

Ha $Tav\,(a,\ b) = Tav\,(x,\ y)$, akkor nem egyértelmű. A két lehetséges esetet mutatja az ábra.
De ha van három olyan a, b és c atomunk, amelyek pozícióját ismerjük, és mindegyik legfeljebb 2 kérdésben szerepelt, akkor ezek közül kettő távolsága biztosan nem egyenlő $Tav\,(x,\ y)$-al. Ez biztosítható kezdetben úgy, hogy meghatározzuk az 1. 2. és 3. atom pozícióját.

```
1  #include <bits/stdc++.h>
2  #include "Meter.h"
3  #define maxN 10001
4  using namespace std;
5  int main(){
6     int N;              //az atomok száma
7     int S[maxN];        //a megoldás
8     int Poz[maxN];      //relatív atom pozíciók
9     int a,b,c,x,y;      //atom címkék
10    int Dab,Dxy,Dax,Dbx,Dby;//távolságok
11    int i,k,minp,z;
12    a=1; b=2; c=3;   //3 atom választása
13    Dab=Span(a,b); Dax=Span(a,c); Dbx=Span(b,c);
14    Poz[1]=0; Poz[2]=Dab;
15    if (Dax==Dab+Dbx || Dab==Dax+Dbx){
16       //c  a->b relatív pozíciójának kiszámítása
17       Poz[c]=Poz[a]+Dax;
18    }else if (Dbx==Dax+Dab){
19       Poz[c]=Poz[a]-Dax;
20    }
21    i=3;
```

```
22      while(i+2<=N){
23  //a,b és c kétszer szerepelt kérdésben és rel. pozí
24        x=i+1; y=i+2; //a következő 2 atom: x, y}
25      i+=2;
26      Dxy=Span(x,y);//x és y távolságának lekérdezése
27  //válasszunk 2 atomot [a,b,c] közül, hogy
28  //távolságuk különböző legyen Dxy-től.}
29      if (Dxy!=abs(Poz[b]-Poz[c])){//{b--c<>x--y}
30        z=a; a=c; c=z;
31      }else if (Dxy!=abs(Poz[a]-Poz[c])){//{a--c<>x--y}
32        z=b; b=c; c=z;
33      }  //else a--b<>x--y
34      Dab=abs(Poz[a]-Poz[b]);
35      if (Poz[b]<Poz[a] ){// Poz[a]<Poz[b] biztosítása
36        z=a; a=b; b=z;
37      };
38      //Poz[a]<Poz[b], Dab<>Dxy
39      Dax=Span(a,x);
40      Dby=Span(b,y);
```

```
41  //x és y relatív pozíciójának meghatározása}
42  if (Dax==Dab+Dby+Dxy ||              //{a--b--y--x}
43     Dax+Dxy==Dab+Dby) {               //{a--b--x--y}
44     Poz[x]=Poz[a]+Dax;                //{a--x--b--y}
45     Poz[y]=Poz[b]+Dby;
46  } else if (Dab==Dax+Dxy+Dby ||  //{a--x--y--b}
47     Dab+Dxy==Dax+Dby){             //{a--y--b--x}
48     Poz[x]=Poz[a]+Dax;                //{a--y--x--b}
49     Poz[y]=Poz[b]-Dby;                //{y--a--b--x}
50                                  //{y--a--x--b}
51  }else if (Dxy==Dab+Dax+Dby){    //{x--a--b--y}
52     Poz[x]=Poz[a]-Dax;
53     Poz[y]=Poz[b]+Dby;
54  } else if (Dby==Dxy+Dax+Dab ||   //{y--x--a--b}
55  Dax+Dab==Dby+Dxy){             //{x--y--a--b}
56     Poz[x]=Poz[a]-Dax;                //{x--a--y--b}
57     Poz[y]=Poz[b]-Dby;
58  };
59  a=x; b=y;                       // a, b helyett x és y
60  }//while
```

```
61  if (N%2==0){//az utolsó elem feldolgozása,ha N páros
62     if (Poz[b]<Poz[a]){z=a; a=b; b=z;}
63  Dab=abs(Poz[a]-Poz[b]);
64  Dax=Span(a,N);
65  Dbx=Span(b,N);
66  if (Dax==Dab+Dbx){
67     Poz[N]=Poz[b]+Dbx;
68  }else if (Dab==Dax+Dbx){
69     Poz[N]=Poz[a]+Dax;
70  }else
71     Poz[N]=Poz[a]-Dax;
72  }
73  minp=Poz[1];
74  for(int i=1;i<=N;i++)
75     if (Poz[i]<minp) minp=Poz[i];
76  minp=-minp+1;
77  for(int i=1;i<=N;i++)
78  //a relatív pozíciók 1..N tartományba léptetése
79     S[Poz[i]+minp-1]=i;
80     for(int i=1;i<=N;i++) Answer(i,S[i]);
81  }
```

Feladat: Többségi elem kiválasztása (CEOI'2001)

Iskolád tanulói két csoportba tartoznak. Tudjuk, hogy az egyik csoportban többen vannak, mint a másikban, ezt nevezzük többségi csoportnak. Ki kell választani egy tanulót, aki a többségi csoporthoz tartozik. Ehhez egyetlen műveletet használhatunk, nevezetesen két tanulótól megkérdezhetjük, hogy ugyanabba a csoportba tartoznak-e. Olyan programot kell írni, amelyik a lehető legkevesebb kérdéssel meghatároz egy többségi csoporthoz tartozó tanulót. A tanulókat sorszámukkal azonosítjuk.

Könyvtár

A `query` modul három művelete használható.

- `int Size()`: a tanulók n számát adja ($5 \leq n < 30\,000$, páratlan szám).
- `int Member(int x, int y)`: 1 az értéke, ha az x és az y sorszámú tanuló ugyanazon csoportba tartozik, egyébként 0.
- `void Answer(int x)`: ezzel a művelettel kell megadni a kiválasztott, többségi csoportba tartozó tanulót.

A megoldás csak akkor fogadható el, ha a $\{1, \ldots, n\}$ halmaz bármely olyan A és B diszjunkt két részre osztása esetén, amely kompatibilis a feltett kérdésekkel, a közölt megoldás a nagyobb elemszámú halmazban van.

Pontozás

Helyes válasz esetén a kapott pontszám: $\max(0, n - k)$, ha a programod k `Member` műveletet hajtott végre.

Megoldás

Jelölje $H = \{1, \ldots, n\}$ a tanulók halmazát.
Azt mondjuk, hogy egy $A \subset H$ **homogén** részhalmaz, ha minden eleme ugyanabba a csoportba tartozik, azaz minden x, y esetén `Member(x,y)=1`.
Azt mondjuk, hogy az $U, V \subset H$ homogén részhalmazok **ellentétesek**, ha bármely $x \in U$ és $y \in V$ esetén `Member(x,y)=0`.
Vegyük észre, hogy ha `Member(x,y)=0`, akkor x és y egyszerre kihagyható H-ból, mert a maradék is biztosan tartalmaz egy többségi elemet. Általánosabban, ha $U \subset H$ és $V \subset H$ azonos elemszámú ellentétes részhalmazok, akkor egyszerre mindkettő kihagyható H-ból.
Bármely kérdéssorozat által nyert ismeret ábrázolható egy olyan halmazzal, amelynek az elemei diszjunkt halmazpárok.
Pontosabban, az alábbi feltételeket kielégítő I halmazzal.

$$I = \{ (U_1, V_1), \ldots, (U_m, V_m) \}$$

$$U_i, V_ij \text{ diszjunktak } 1 \leq i, j \leq m$$

$$U_i \text{ és } V_i \text{ homogén } 1 \leq i \leq m$$

$$U_i \text{ és } V_i \text{ ellentétesek } 1 \leq i \leq m$$

$$\bigcup_{i=1}^{m} (U_i \cup V_i) = \{1, \ldots, n\}$$

A kezdeti helyzetet, amikor nincs semmi ismeretünk, az alábbi halmaz ábrázolja.

$$I = \{ (\{1\}, \emptyset), \ldots (\{n\}, \emptyset) \} \tag{1}$$

Tegyük fel, hogy az eddig elvégzett kérdésekkel nyert információt az I halmaz ábrázolja, és a `Member(x,y)` kérdést tesszük fel, ahol x és y nem ugyanazon homogén részhalmazban van. Mivel az I-beli részhalmazok páronként diszjunktak, feltehetjük, hogy

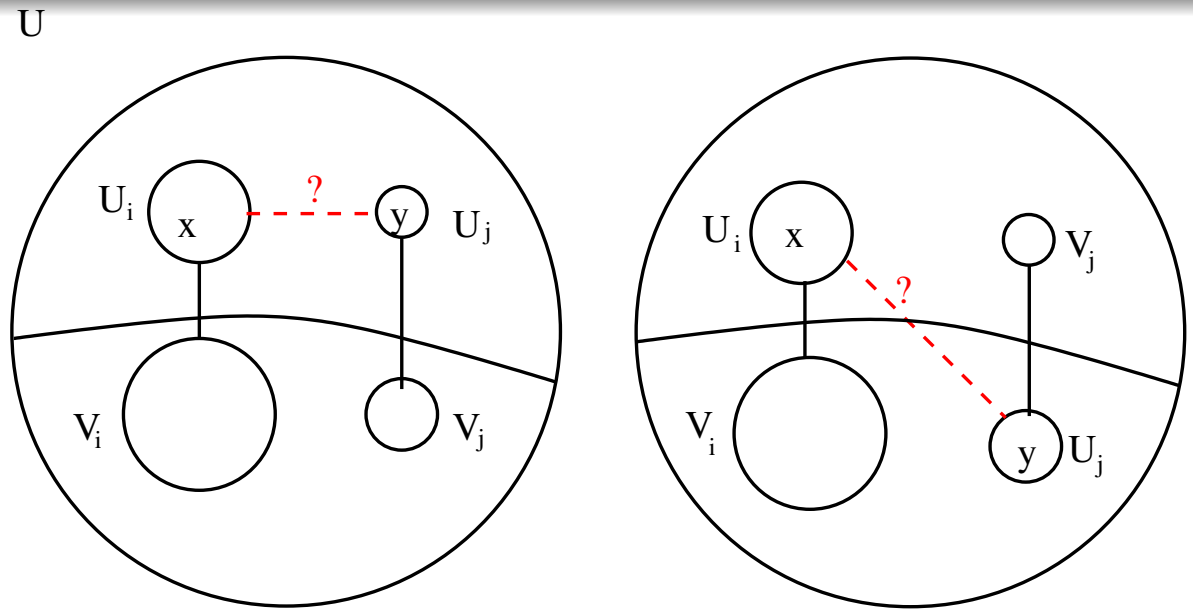
$$x \in U_i \text{ és } y \in U_j$$

Ha `Member(x,y)=1`, akkor az (U_i, V_i) és (U_j, V_j) párt töröljük és vegyük be helyettük az alábbi (U, V) halmazpárt

$$U = U_i \cup U_j \quad (2)$$
$$V = V_i \cup V_j \quad (3)$$

Ha pedig `Member(x,y)=0`, akkor az (U_i, V_i) és (U_j, V_j) párt töröljük és vegyük be helyettük az alábbi (U, V) halmazpárt

$$U = U_i \cup V_j \quad (4)$$
$$V = U_j \cup V_i \quad (5)$$



5. ábra

Nyilvánvaló, hogy bármely x akkor és csak akkor fogadható el a feladat helyes megoldásának, ha az elvégzett kérdésekhez tartozó I ismeretre teljesül, hogy ha $x \in U_k$ akkor

$$|U_k| + \sum_{i=1, i\neq k}^{m} \min(|U_i|, |V_i|) \quad > \quad |V_k| + \sum_{i=1, i\neq k}^{m} \max(|U_i|, |V_i|$$

Észrevétel: a `Member(x,y)` kérdéseket szervezhetjük úgy, hogy x és y azonos elemszámú, különböző részhalmazhoz tartozik, amely ellentettjei üres halmazok. Tehát a kérdés hatására `Member(x,y)=1` esetén összevonjuk a két részhalmazt, az ellentettjük marad üres. `Member(x,y)=0` esetén pedig mindkét részhalmazt eltávolítjuk. Ebből következik, hogy az I ismeret minden kérdés után olyan, hogy a párok egyik tagja üres, és a másik elemszáma kettő-hatvány. Mivel minden részhalmazból elég egy reprezentáns elemet ismerni, ezért az I ismeret ábrázolható az R és B sorozattal. Ahol $R[i]$ azon részhalmaz egy eleme, amelynek elemszáma 2^i, $B[i]$ pedig akkor és csak akkor igaz, ha van 2^i elemszámú részhalmaz I-ben. Ha L a legnagyobb index, hogy $B[L]$ igaz, akkor az alábbi egyenlőtlenség alapján $R[L]$ a többségi elem, ha

$$2^L > \sum_{i=0}^{L-1} B[i] * 2^i$$

Ha M az aktuális elemszám (ami a törlések után maradt), akkor az egyenlőtlenség ekvivalens a következővel.

$$2^L > M/2$$

megvalósítás

```
1  #include <bits/stdc++.h>
2  #include "query.h"
3  #define maxK 21
4
5  int  N;                  //elemszám
6  int  M;                  //aktuális elemszám
7  bool B[maxK];            //van 2^k elemszámú részcsoport
8  int  R[maxK];            //R[k] a reprezentáns
9  int  Pow2[maxK];         //Pow2[k]=2^k
10 int  L,i,k;//2^L a legnagyobb részcsoport elemszáma
```

```
1    int main(){
2      Pow2[0]=1;
3      for(int i=1; i<maxK;i++)
4      Pow2[i]=2*Pow2[i-1];
5      N=Size(); M=N;
6      L=0;  i=0;
```

```
7         while(i<N){//rákérdezés az R[k], i párra
8           k=0; B[0]=true;
9           i++; R[0]=i;  i++;//i=R[0]
10          if (i>N) break;
11          while (B[k]){      //van 2^k elemű részcsoport
12            if(Member(R[k],i)==1){// összevonása
13              B[k]=false;
14              k++;
15              if (k>L) L=k;//új legnagyobb részcsoport
16            }else {              //R[k] és i ellentétes
17              M-=Pow2[k+1];//M aktualizálása
18              B[k]=false;   //törlés
19              if (k==L)     //L aktualizálása
20              while (L>0 && !B[L]) L--;
21              k=-1; break;
22            };
23          }//while B[k];
24          if (k>=0) {
25            B[k]=true;      //új 2^k elemű részcsoport
26            R[k]=i;         //R[k] reprezentánsa i
27          };
```

```
28
29          if (L>0 && Pow2[L]>M/2)
30          //a legnagyobb részcsoport többségi
31          break;
32        }//while i<N;
33
34        Answer(R[L]);                    //a megoldás
35      }
```

A megoldás legrosszabb esetben

$$n - b(n) \tag{7}$$

`Member()` műveletet hajt végre, ahol $b(n)$ az n szám bináris leírásában az 1-es bitek száma.
Bizonyítható, hogy ennél kevesebb kérdéssel nem lehet kitalálni.

Feladat: Ádám és Éva kitalálós játéka

Ádám és Éva kitalálós játékot játszik. Éva gondol egy 1 és N közötti egész számot, amelyet Ádámnak ki kell találnia. Ádám olyan kérdést tehet fel, hogy "A gondolt szám kisebb-e, mint x?". Éva válasza "igen", vagy "nem" lehet. Hogy a játék érdekesebb legyen, megállapodtak abban, hogy Ádám legfeljebb h-szor tehet fel olyan kérdést, amelyre a válasz "nem", tehát ha már h kérdésére "nem" választ kapott, akkor tovább nem kérdezhet.
Írj programot, amely a lehető legkevesebb kérdéssel kitalálja, hogy Éva milyen számra gondolt!
Vigyázat! Éva csalfa válaszadó. Mindig úgy válaszol, hogy válasza kompatibilis legyen azzal, amit Ádám eddig kérdezett, de a legtöbbet kelljen kérdeznie.

Könyvtár

A `valasz` modul az alábbi négy műveletet tartalmazza.

`int getN()` : Pontosan egyszer kell hívni a program elején, és a visszaadott n pozitív egész szám azt jelenti, hogy az Éva által gondolt szám legfeljebb n ($1 < n \leq 100\,000\,000$)

`int getH()` : Azt a H ($1 \leq H \leq 32$) számot adja, ahányszor nemleges választ adó kérdést lehet feltenni.

`int Kerdes(int x)` : A felteendő kérdést valósítja meg: ha a gondolt szám kisebb, mint x, akkor 1, egyébként 0 értéket ad.

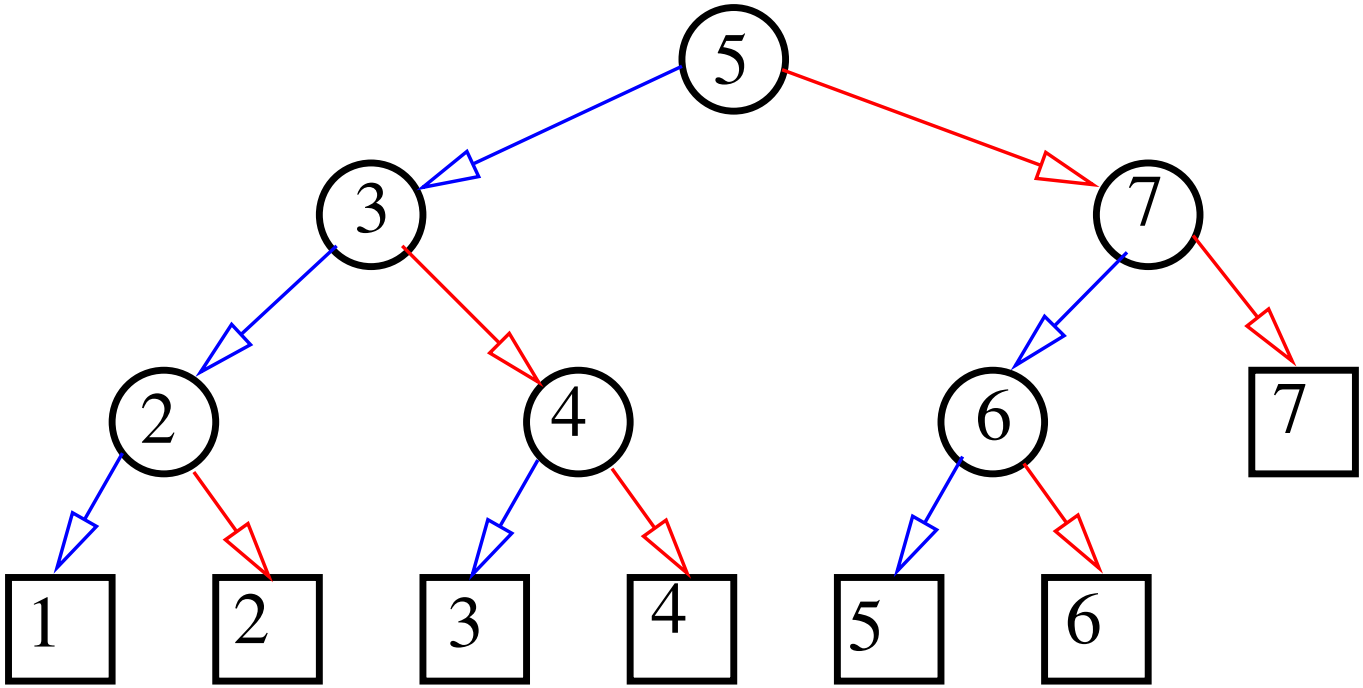
`void Megoldas(int x)` : A kitalált számot ezzel a művelettel kell közölni, a kitalált szám az x argumentum. Blöffölni nem lehet! Csak akkor fogadható el a közölt x érték, ha a feltett kérdések alapján a gondolt szám csak x lehet.

A programod tartalmazza a `eva.h` importáló sort!

Gyakorlás

Az értékelő szerverről letölthető minta.zip állomány tartalmaz egy forrásnyelvű (C/C++) ellenfél modult (eva.cpp, eva.h). Ezt a két fájlt hozzá kell venni a megoldásodat tartalmazó projekthez! Ez csak üres eljárásokat tartalmaz, szabadon módosíthatod. Saját gépen próbáláskor a standard bemenet első sorába két egész számot kell írni egy szóközzel elválasztva, az N és a H értékét!

Megoldás

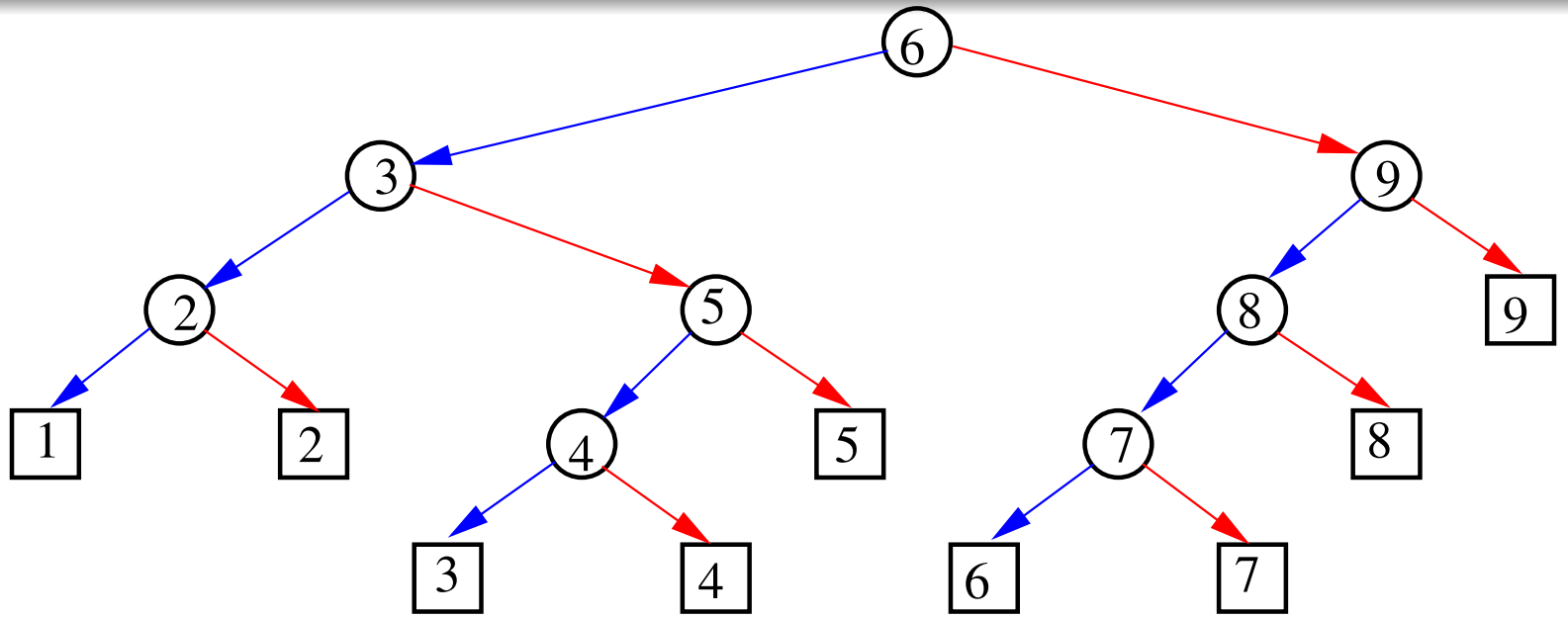


6. ábra

Ha nincs korlát a hibázásra (H nagy), akkor a jól ismert bináris keresés a megoldás.
Ekkor a fa magassága $\lceil \log 2(N) \rceil$.
A jobb-magassága, a hibázási korlát $\lfloor \log 2(N) \rfloor$

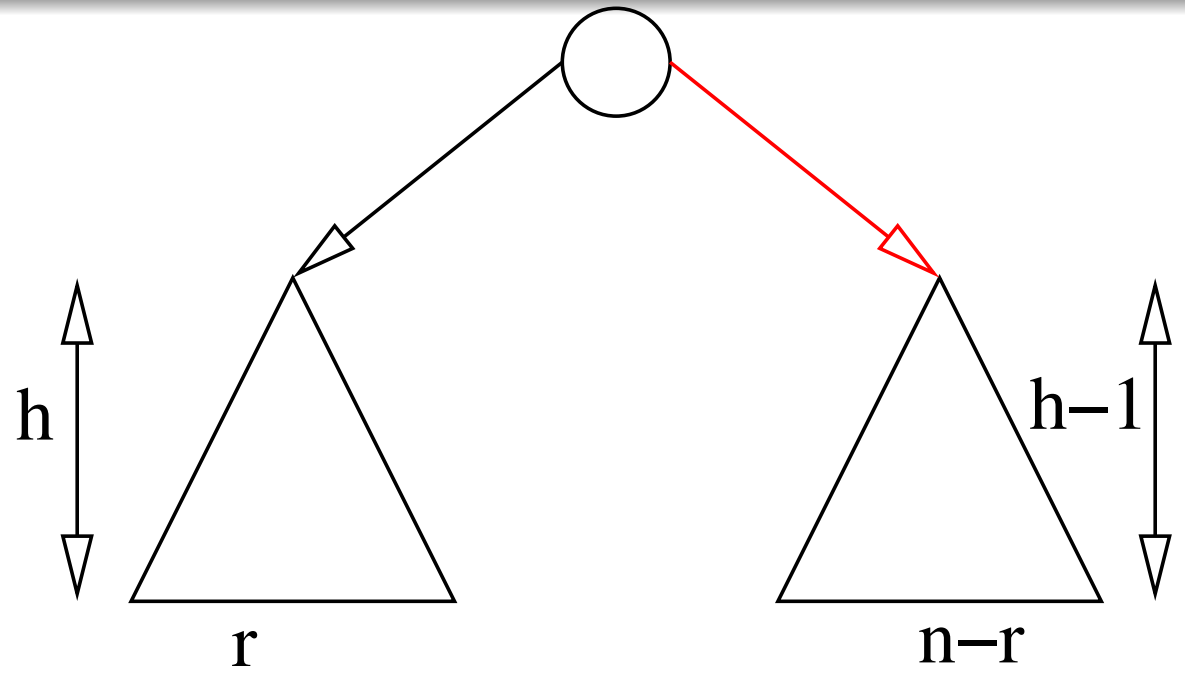
Minden olyan bináris fa, amely teljesíti az alábbi három feltételt, kifejez egy olyan kérdezési stratégiát, amely során legfeljebb h kérdésre kaphatunk nem választ. Fordítva is igaz, tehát minden olyan kérdezési stratégia, amely során legfeljebb h kérdésre kaphatunk nem választ, kifejezhető ilyen fával.

1. A fának n levele van és ezek balról jobbra sorrendben az $1, \ldots, n$ számokat tartalmazzák.
2. A fának $n-1$ belsőpontja van. Minden p belső pont a p jobb-részfájában lévő levél értékek minimumát tartalmazza.
3. Bármely gyökértől levélig vezető úton legfeljebb h-szor megyünk jobbra. (h a jobb-magasság értéke.)



7. ábra. 2-hibázó kérdezőfa 9 számra.

Tehát legfeljebb annyi kérdést kell feltenni, amennyi a fa magassága. Ezért állítsuk elő azt a h-hibázó kérdezőfát, amelynek n levele van és a magassága a lehető legkisebb.
Jelölje $M(n, h)$ a legkisebb magasságú olyan h hibázó kérdezőfa magasságát, amelynek n levele van.
$M(n, h)$ kiszámítható dinamikus programozás módszerével az alábbi rekurzív összefüggések alapján.
Jelölje $R(n, h)$ az $M(n, h)$ magasságú h-hibázó kérdezőfa bal-részfájában a levelek számát.



8. ábra

$$M(n,\,1) = n-1 \tag{8}$$

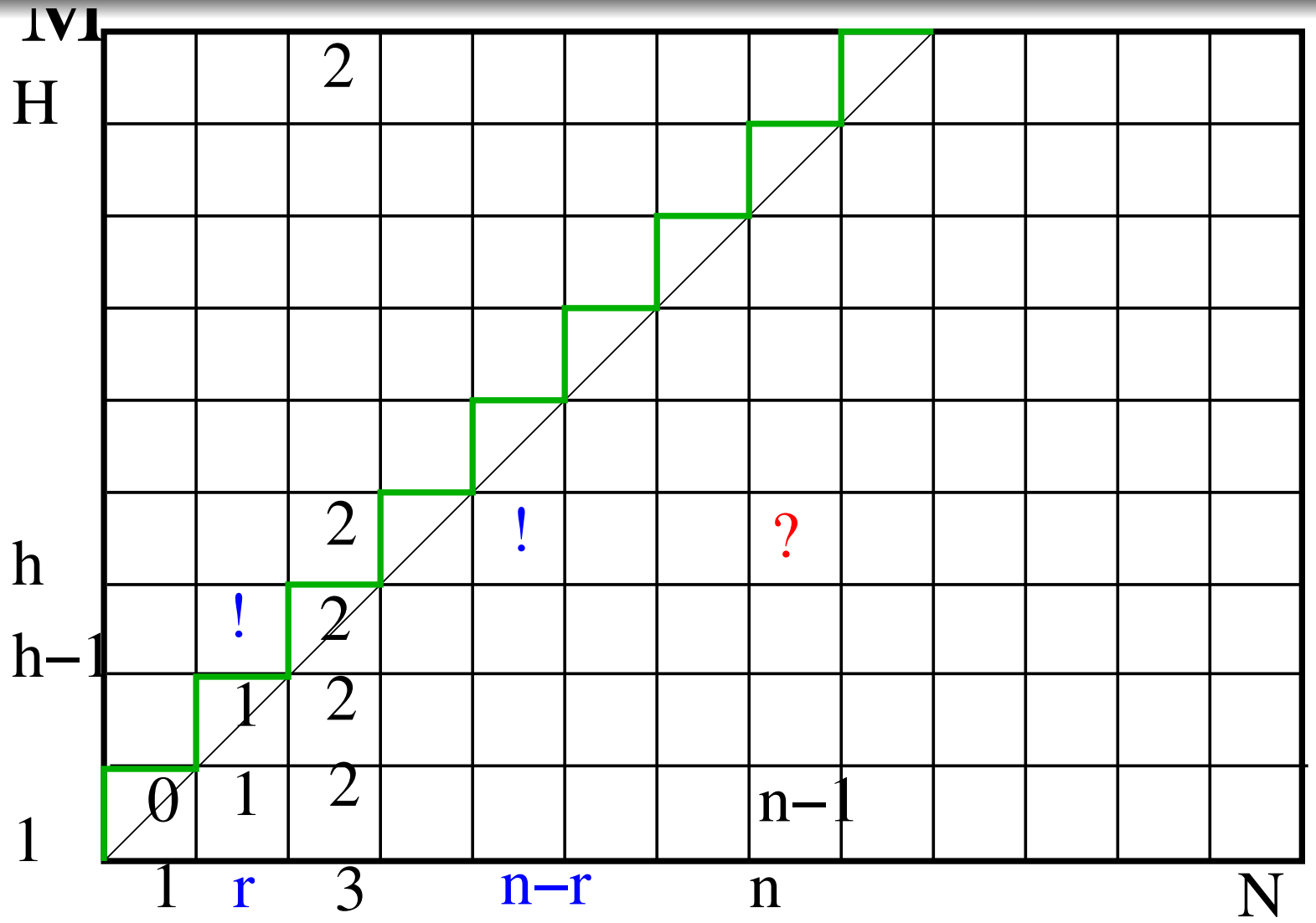
$$R(n,\,1) = n-1 \tag{9}$$

$$M(n,\,h) = \lceil \log 2(N) \rceil \text{ ha } \lfloor \log 2(N) \rfloor \leq h \tag{10}$$

$$R(n,\,h) = (n+1)/2 \text{ ha } \lfloor \log 2(N) \rfloor \leq h \tag{11}$$

$$M(n,\,h) = 1 + min_{r=1}^{n-1}\,(max\,(M(r,\,h),\,M(n-r,\,h-1)) \tag{12}$$

$$R(n,\,h) = r : max\,(M(r,\,h),\,M(n-r,\,h-1)) \text{ minimális} \tag{13}$$



9. ábra

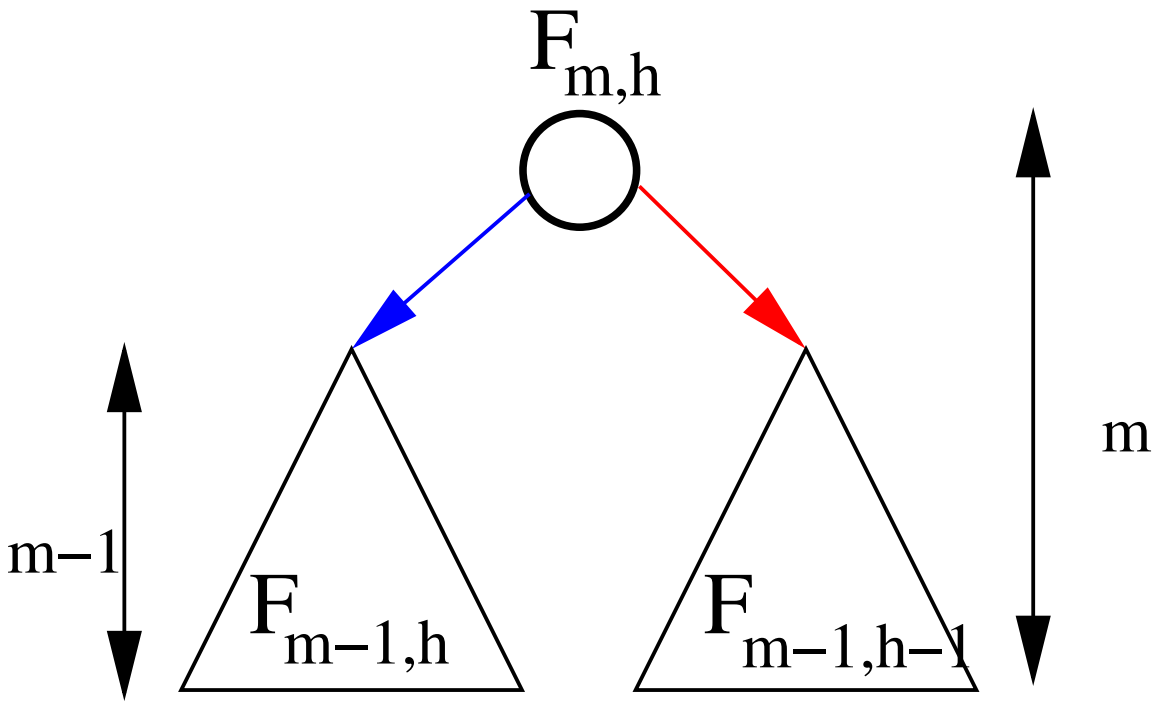
```
1     Elofeldolgozas();
2     int bal=1, jobb=N, n=N, h=H, k;
3     while(bal<jobb){
4        n=jobb-bal+1;
5        if(Mt[n][h]<0) M(n,h);
6        k=bal+Rt[n][h];
7        if(Kerdes(k)==1){
8           jobb=k-1;
9        }else{
10          bal=k;
11          h--;
12       }
13    }
14    Megoldas(bal);
```

$M(n, h)$ kiszámítása rekurzió-memorizálással.

```
1     int M(int n, int h){
2        if(Mt[n][h]>=0) return Mt[n][h];
3        int mn=ceil(log2(n));
4        int hn=floor(log2(n));
5        if(hn<=h){
```

```
6           Mt[n][h]=mn;
7           Rt[n][h]=(n+1)/2;
8           return mn;
9         }
10        int m=n, r=n-1, mk;
11        for(int k=1;k<n;k++){
12          mk=max(M(k,h),M(n-k,h-1));
13          if(mk<m){
14            m=mk; r=k;
15          }
16        }
17        Rt[n][h]=r; //memorizálás
18        Mt[n][h]=m+1;
19        return m+1;
20      }
```

Azonban n értéke túl nagy lehet ahhoz, hogy $M(n, h)$ és $R(n, h)$ értékét táblázat-kitöltéssel ki tudjuk számítani.
Mivel h értéke nem nagy, ezért minden m magasságra ki tudjuk számítani, hogy legjobb esetben hány levele van a h-hibázó, m magasságú fának. Jelölje ezt $L(m, h)$, a megfelelő kérdezőfát pedig $F_{m,h}$. Tehát adott n-hez keressük azt a legkisebb m-t, hogy $L(m, h) \geq n$.



10. ábra. m magas, h-hibázó legtöbb levelet tartalmazó fa.

$$L(m, 1) = m + 1 \quad (14)$$

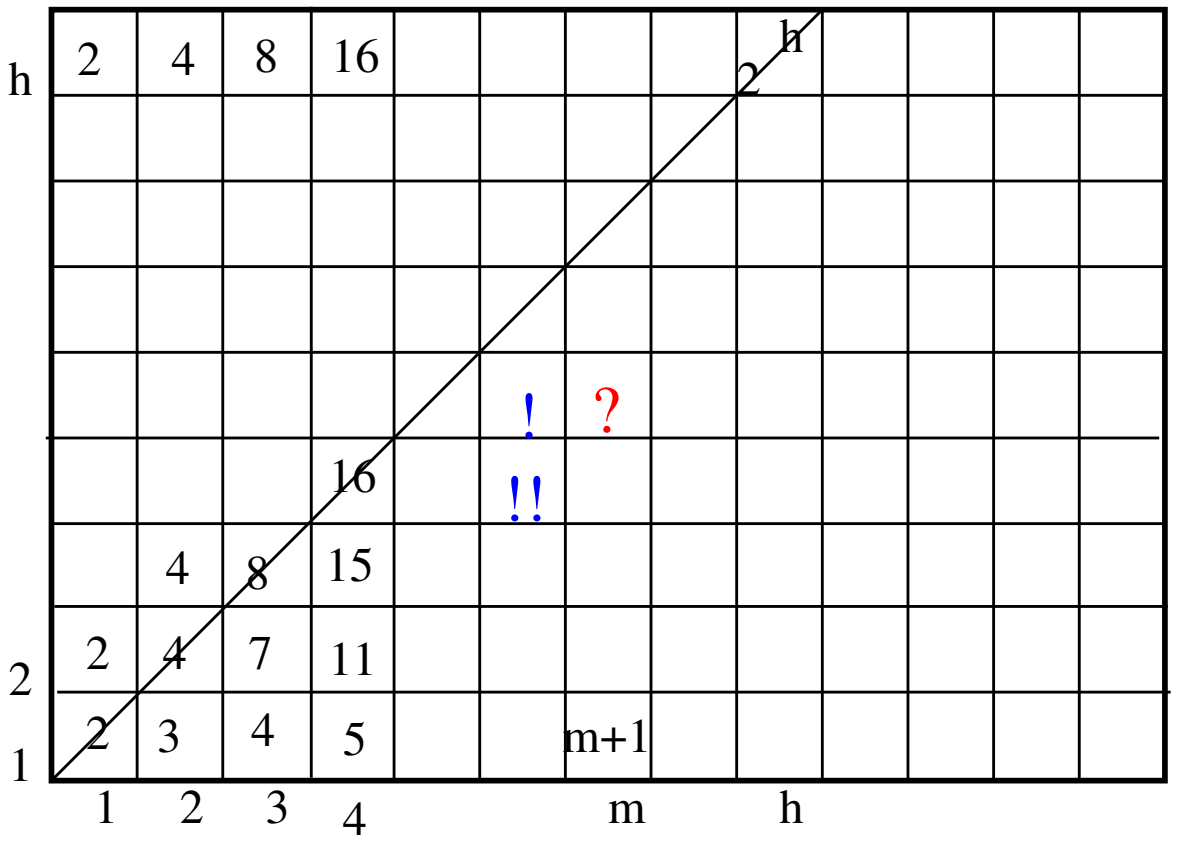
$$L(m, h) = L(m, m) \text{ ha } m < h \quad (15)$$

$$L(m, h) = L(m-1, h) + L(m-1, h-1)\, h > 1,\ m \geq h \quad (16)$$

Ha ismerjük $L(m, h)$ értékeket, akkor a megoldás, feltéve, hogy m a legkisebb olyan magasság, hogy $L(m, h) \geq n$

```
1   Elofeldolgoz();
2   int bal=1, jobb=n, k;
3   while(bal<jobb){
4     k=bal+L[m][h];
5     if(kerdes(k)==1)
6     jobb=k-1;
7     else{
8       bal=k;
9       h--;
10    }
11    n=jobb-bal+1;
12    while(L[m][h]>=n) m--;
13    if(h>m) h=m;
14  }
```

```
15      megoldas(bal);
```



11. ábra. Az `L[][]` táblázat-kitöltés felülről lefelé haladva.

Vegyük észre, hogy ha ismerjük $L(m, x)$ érékét minden $x = 1, \ldots, h$-ra, akkor

$$L(m-1, x) = L(m, x) - L(m-1, x-1)$$

Tehát elég egy oszlopot tárolni az L táblázatban, $L[m][]$ értékekből $L[m-1][]$ kiszámítható (O(h) időben).

```
1   #include <bits/stdc++.h>
2   #include "valasz.h"
3   #define maxH 32
4   using namespace std;
5   int L[maxH];
6   int n,h,m;
7   void Elofeldolgoz () {
8     int i,hi;
9     m=1; for(int i=1;i<=h;i++) L[i]=2;
10    while(L[h]+L[h-1]<n){
11      m++;
12      for(int hi=h; hi>1; hi--)
13      if(m<=h)
14      L[hi]=2*L[hi];
15      else
16      L[hi]+=L[hi-1];
17      L[1]=m+1;
18    }
19  }//Elofeldolgoz
```

```
1   int main (){
2     n=getN();
3     h=getH();
4     Elofeldolgoz();
5     int bal=1, jobb=n, k,nn;
6     while(bal<jobb){
7       k=bal+L[h];
8       if(kerdes(k)==1)
9       jobb=k-1;
10      else{
11        bal=k;
12        h--;
13      }
14      nn=jobb-bal+1;
15      for(;;){
16        m--;
17        L[1]=m+1;
18        if(h>m) h=m;
19        for(int hi=2;hi<=h; hi++)
20        L[hi]-=L[hi-1];
21        if(L[h]<nn) break;
```

```
22            }
23        }
24        megoldas(bal);
25    }
```

Feladat: Medián (IOI'2000)

Egy űrkísérletben n tárgyat használunk, melyeket 1-től n-ig számozunk, ahol n páratlan. Minden tárgy különböző súlyú (természetes számok), de magukat a súlyokat nem ismerjük. Minden Y súlyra igaz, hogy $1 \leq Y \leq n$. Mediánnak nevezzük azt a tárgyat, amelyiknél ugyanannyi könnyebb, mint nehezebb tárgy van. Írj programot, amely meghatározza a mediánt! A tárgyak súlyát olyan eszközzel hasonlíthatjuk össze, amely három tárgy közül megadja a mediánt (a középsőt).

Könyvtár

A `device` nevű könyvtárból az alábbi három művelet használható:

`int GetN()`: Egyszer kell meghívni, a programod legelején; az argumentum nélküli függvényhívás eredménye az n értéke.

`int Med3(int a, int b, int c)`: Három különböző tárgy sorszámával kell hívni, függvényértéke e három sorszám közül a mediánjuk sorszáma.

`Answer(int m)` Egyszer kell meghívni, a programod végén; argumentumként az N tárgy mediánjának a sorszámát kell megadnod. Ez a hívás le is állítja a programodat.

Gyakorlás

A program a standard bemenetről olvassa az adatokat. Az elsö sorba a tárgyak számát (N) kell írni. A második sor a tárgyak súlyát tartalmazza, 1 és N közötti különböző egész számokat, ahol az i-edik érték az i-edik tárgy súlya.

Példa

```
5
2 5 4 3 1
```

Ez az állomány a következő esetet írja le:

Sorszám	1	2	3	4	5
Súly	2	5	4	3	1

Ebben az esetben egy helyes hívási sorrend:

1. GetN() értéke 5
2. Med3(1,2,3) értéke 3
3. Med3(3,4,1) értéke 4
4. Med3(4,2,5) értéke 4
5. Answer(4)

Feltételek

$5 \leq n \leq 1499$ és n páratlan.

Minden i sorszámra igaz: $1 \leq i \leq N$.

Minden Y súlyra igaz: $1 \leq Y \leq N$ és minden súly különböző.

Futtatásonként a `Med3` legfeljebb 7777-szer hívható.

Programod nem olvashat és nem írhat egyetlen állományt sem.

Megoldások

Hagymahámozó algoritmus.

Alapelv: ismételten határozzuk meg és távolítsuk el a két szélső elemet.
A $H \subseteq \{1, \ldots, n\}$ halmaznak $a, b \in H$ két szélső eleme, ha minden $x \in H$ elemre, ha $x \neq a$ és $x \neq b$

$$Med3\,(a, x, b) = x$$

```
1  int S[maxN] ;
2  int  l,r,x,y,mi;
3  for(int x=1;x<=N;x++) S[x]=x;
4  l=1; r=N;
5  while(l<r){
6    for(int x=l+1;x<r;x++){
7      mi=Med3(S[l],S[x],S[r]);
8      if(mi==S[l])
9        Csere(l,x);
10     else if(mi==S[r])
11       Csere(r,x);
12   }//for
13   l++; r--;
14 }
15 Answer(S[l]);
```

A két szélső elem $n-2$ kérdéssel határozható meg, tehát a kérdések száma:

$$(n-2) + (n-4) + \ldots + 3 + 1 = \left(\frac{n-1}{2}\right)^2$$

Ha $n \leq 177$, akkor legfeljebb 7744 hívás kell, de ha $n \geq 179$, akkor legalább 7921.

Egy $\langle a_1, a_2, \ldots, a_m \rangle$ elemsorozatot rendezettnek nevezünk, ha

$$(\forall i, j, k)\ (1 \leq i < j < k \leq m)\ (Med3\,(a_i, a_j, a_k) = a_j)$$

Definiálhatnánk egy $x < y$ bináris lineáris rendezési relációt, amely lehetővé tenné, hogy bármely ismert rendezést használhassunk. A lehetséges esetek, amikor $x < y$:

$$x, y, 1, 2$$
$$x, 1, y, 2$$
$$x, 1, 2, y$$
$$1, x, y, 2$$
$$1, x, 2, y$$
$$1, 2, x, y$$

Ehhez azonban két Med3 hívás kell.

Rendezett sorozat előállítható ismételt beszúrással, indulva egy kételemű sorozattal.

A beszúrás helye meghatározható:

Lineáris kereséssel. A beszúrandó x elemet a sorozat két utolsó eleméhez hasonlítjuk

$\texttt{Med3}(a_{m-1}, x, a_m)$ lekérdezéssel.

$\langle a_1, a_2, \ldots, a_{m-1}, a_m \rangle$

Bináris kereséssel. A beszúrandó x elemet a sorozat középső két eleméhez hasonlítjuk; $\texttt{Med3}(a_{k-1}, x, a_k)$ lekérdezéssel.

$\langle a_{bal}, \ldots, a_{k-1}, a_k, \ldots, a_{jobb} \rangle$

Harmadoló kereséssel.

A beszúrandó x elemet a sorozat egy-harmad és kétharmad pozíciójában lévő két eleméhez hasonlítjuk; $\texttt{Med3}(a_k, x, a_l)$ lekérdezéssel.

$\langle a_{bal}, \ldots, a_k, \ldots, a_l, \ldots, a_{jobb} \rangle$

1. Teljes rendezés algoritmus.

Képezzünk az $\{1, \ldots, n\}$ elemekből rendezett sorozatot:

$$S = \langle a_1, \ldots, a_n \rangle$$

A keresett medián a_m, $m = (n+1)/2$.

2. Felét rendező algoritmus.

Legyen $m = (n+1)/2$. Első lépésként állítsunk elő m elemből pl. az $\{1, \ldots, m\}$ elemekből rendezett sorozatot:

$$S = \langle a_1, \ldots, a_m \rangle$$

Minden további x elemet szúrjuk be az S sorozatba és hagyjuk el az utolsót. A keresett medián a sorozat utolsó eleme lesz.

3. Szűkítő rendezés algoritmus.

Legyen $m = (n+1)/2$. Első lépésként állítsunk elő m elemből pl. az $\{1, \ldots, m\}$ elemekből rendezett sorozatot:

$$S = \langle a_1, \ldots, a_m \rangle$$

Minden további x elemet szúrjuk be az S sorozatba

$$\overline{S} = \langle a_1, \ldots, a_i, x, a_{i+1}, \ldots, a_m \rangle$$

Nyilvánvaló, hogy $\overline{S}$ első eleme nem lehet a medián, mert van legalább m nálánál nagyobb elem. Hasonlóképpen $\overline{S}$ utolsó eleme sem lehet a medián, mert van legalább m nálánál kisebb.

Tehát hagyjuk el $\overline{S}$ első és utolsó elemét.

Beszúrási mód	Változat	Legrosszabb eset	Legjobb eset
Lineáris	Teljes	561749	282532
	Fél	421499	169655
	Szűkítő	281623	141676
Bináris	Teljes	12953	11680
	Fél	12477	11492
	Szűkítő	11481	10471
Harmadoló	Teljes	9399	8977
	Fél	9399	8522
	Szűkítő	8319	8041

1. táblázat. Med3 hívások száma $N = 1499$ elemre

Iterált harmadoló algoritmus.

Elvi algoritmus:
Legyen $H = \{1, 2, \ldots, n\}$. Válasszunk két különböző tetszőleges $a, b \in H$ elemet, ezek lesznek a felosztó elemek. Majd H többi elemét osszuk három részbe:

$$H_1 = \{x : Med3(a, b, x) = a\} \quad (17)$$

$$H_2 = \{x : Med3(a, b, x) = x\} \quad (18)$$

$$H_3 = \{x : Med3(a, b, x) = b\} \quad (19)$$

Tehát a keresést H_1, H_2, H_3 valamelyikében kell folytatni attól függően, hogy hány elem került az egyes halmazokba a felosztás során.

$$H_1 < a < H_2 < b < H_3$$

```
1    void Feloszt(Halmaz H, H1,H2,H3, int a,b){
2    int m;
3    Valaszt(H,a); Valaszt(H,b);
4    Rendez(a,b); //a<b legyen
5    H1=Ures; H2:=Ures; H3:=Ures;
6    for(int x:H){
7       if(x==a || x==b) continue;
8       m=Med3(a,x,b)
9       switch (m){
10         case x: Halmazba(H2,x); break;
11         case a: Halmazba(H1,x); break;
12         case b: Halmazba(H3,x); break;
13      }
14   }
15 }
```

```
16  int Keres(Halmaz H, int k){//H-ban a k. elemet kere
17    Halmaz H1,H2,H3; int a,b;
18    if(H.size()==1){
19      return H[0];
20    }else if(H.size()==2){
21      Rendez(a,b);
22      if(k==1) return a; else return b;
23    }
24    Feloszt(H,H1,H2,H3, a, b);
25    if (k<=H1.size()){
26      return Keres(H1,k);
27    }else if( k==H1.size()+1){
28      return a;
29    }else if (k>|H1|+1) And (k<=|H1|+|H2|+1) {
30      return Keres(H2,k-H1.size()-1);
31    }else if(k==H1.size()+H2.size()+1){
32      return b;
33    }else{
34        return Keres(H3,k-(H1.size()+H2.size()+2));
35      }
36    }//Keres
```

```
37    void main{
38      int N=GetN;
39      Halmaz H;
40      for(int x=1;x<=N;x++) Halmazba(H,x);
41      Answer(Keres(H, (N+1)/2));
42    }
```

A legrosszabb esetben ez a megoldás nem jobb, mint a hagymahámozó.
Azonban, a két felosztó elem véletlen választásával igen nagy valószínűséggel a Med3 kérdések száma a korlát alatt marad.

Kupacot használó algoritmusok.

Tegyük fel, hogy van olyan "Kupac" adattípusunk, amely az alábbi műveleteket biztosítja:

KupacEpit(V): a V halmaz elemeiből kupacot képez

CsereMin(H, x): a H kupac legkisebb elemét kicseréli x-re, ha $x > \texttt{min}(H)$ egyébként H nem változik.

KupacMin(H): visszaadja $\texttt{min}(H)$-t.

Ezeket a műveleteket felhasználva kapjuk az alábbi elvi algoritmust:

$N = GetN$
$m = \lceil N/2 \rceil$
$H = KupacEpit(\{1, \ldots, m\})$
teljesül: $|\{1 \leq i \leq m : \min(H) < i\}| = m - 1$
for($int\ x = m + 1, x \leq N, x++$ **){**
  $CsereMin(H, x)$
  invariáns: $\{1 \leq i \leq x : \min(H) < i\} = m - 1$
}
teljesül: $|\{1 \leq i \leq N : \min(H) < i\}| = m - 1 =$
$= |\{1 \leq i \leq N : \min(H) > i\}|$
$Answer\ (\ KupacMin(H)\ )$

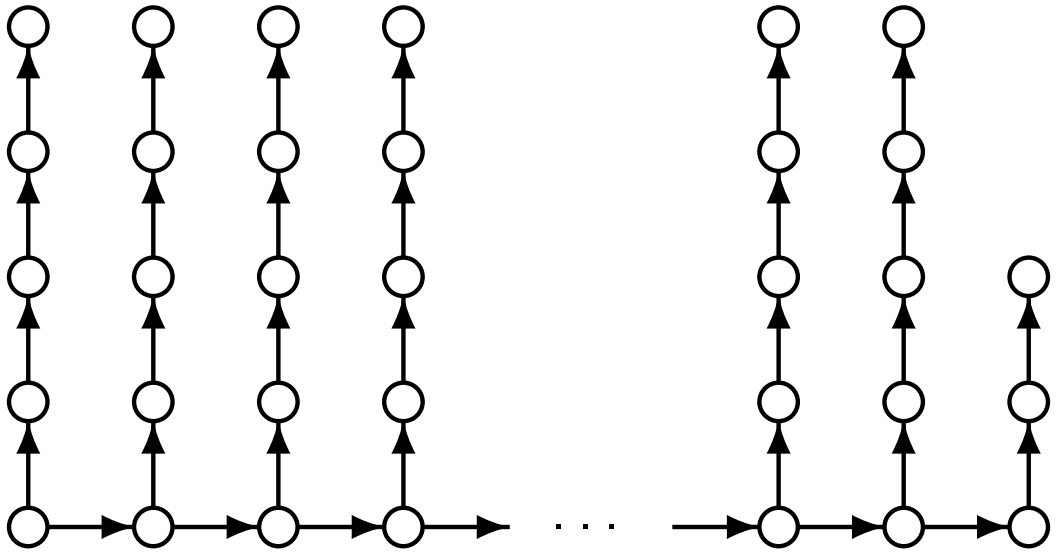
Egy nyilvánvaló megvalósítás lehet a rendezett sorozat (tömb). Ekkor a *CsereMin* művelet $\lceil \log_3(m+1) \rceil$ Med3 hívást igényel. A *KupacEpit* pedig legrosszabb esetben

$$C(m) = 3 + \sum_{i=4}^{m} \lceil \log_3(i) \rceil \qquad (20)$$

A kupac megvalósítható lenne standard bináris kupacként is, azonban ekkor, a bináris rendezés két `med3` műveletet igényel, így a kupacépítés nélkül is $2 \cdot lg_2(n/2) \cdot n/2 = 15\,000$ `med3` művelet kellene legrosszabb esetben.

2-dimenziós rendezett listán alapuló algoritmus.

A kupac megvalósításához nem kell teljesen rendezett sorozat! A szükséges kupac műveletek megvalósíthatók 2-dimenziós rendezett lista alkalmazásával is.



12. ábra. 2-dimenziós rendezett lista

Kupacépítés

Az $1, \ldots, m = (N+1)/2$ elemekből készítsünk $v_1, \ldots v_k$ függőleges rendezett sorozatot.

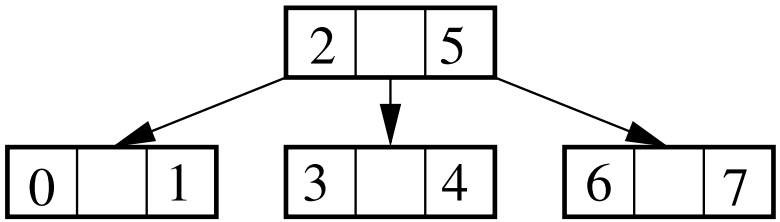
A v_i sorozatokat rendezzük az első elemük szerint növekvően:

$L = (i1, \ldots, i_k)$

Ekkor v_{i1} első eleme a legkisebb az $\{1, \ldots m\}$ elemek között.

A *CsereMin*(H, x) művelet a következőképpen végezhető:

Szúrjuk be x-et a v_{i1} függőleges listába, töröljük v_{i1} első elemét, majd a v_{i1} listát szúrjuk be az L listába. A beszúrást mindkét esetben harmadoló kereséssel végezzük.



13. ábra. 8 elemű rendezett sorozatba 2 Med3 kérdéssel beszúrható új elem.

Látható, hogy minden olyan rendezett sorozatba, amelynek az elemszáma $h = 3^k - 1$, egy új elem k `med3` művelettel beszúrható.

Tehát érdemes a függőleges láncok h elemszámát úgy választani, hogy $h = 3^k - 1$ legyen. Azt kapjuk, hogy ha $h = 8$ legfeljebb 6825 `med3` hívás kell, ha $N = 1499$. A $h = 26$ esetén is teljesül a 7777 feltétel.

Megjegyezzük, hogy ha $h = \lceil \sqrt{m} \rceil$ (és ekkor $k = \lfloor \sqrt{m} \rfloor$) nem teljesül a 7777 feltétel.

Feladat: Centrum

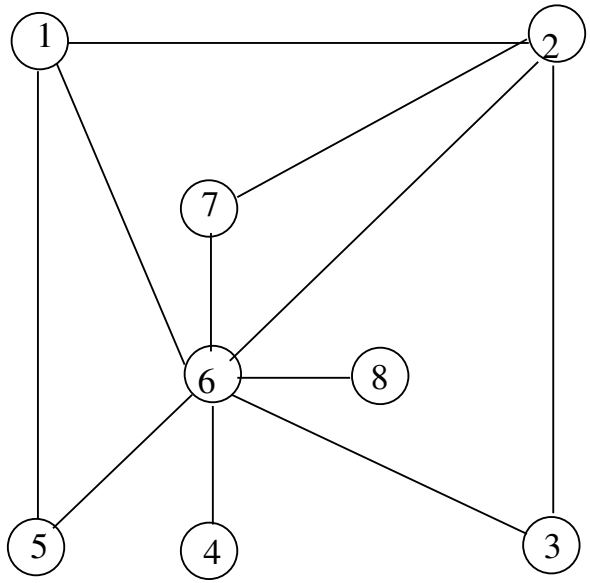
Nagyváros metróhálózata N állomást tartalmaz. Ismert, hogy mely állomáspárok között közlekedik metró mindkét irányban. A metró hálózat olyan szerkezetű, hogy van egy állomás, amelyiket vonal köt össze minden más állomással, ez a **centrum**. Továbbá, van legalább egy olyan állomás, amely csak a centrummal van összekötve vonallal. A centrum állomásnál van találkozód a barátoddal, de nem tudod, hogy melyik a centrum. Van egy speciális tudakozó, amely kérésre megmondja, hogy adott A és B állomás között van-e vonal. Egy kérés költsége egy érme, és pontosan $2N$ érméd van, tehát legfeljebb $2N$ kérdéssel kell kitalálni, hogy melyik a centrum. Készíts programot, amely kitalálja, hogy melyik a centrum! A program megvalósításához a `tudakozo` könyvtár műveleteit kell használni.

`int hany(void);` – az állomások N számát adja ($2 \leq N \leq 10000$). Az állomásokat az $1, \ldots, N$ számokkal azonosítjuk. Ennek a műveletnek a végrehajtása meg kell hogy előzze az első kérdés műveletet!

`int kerdes(int x, int y);` – a művelet két paramétere két különböző állomás sorszáma legyen: x y. A művelet egy egész számot ad vissza, amely 1, ha az x és y állomást vonal köti össze, egyébként 0.

`void megoldas(int x);` –a művelet egyetlen paramétere a centrum sorszáma legyen. Végrehajtásával a program végrehajtása normálisan befejeződik. Csak olyan választ fogadunk el, amely a feltett kérdések alapján helyes, tippelni nem lehet!
A válaszadó adaptív.

Letölthető egy minta `tudakozo` modul programja. A `tudakozo` a standard bemenetről három egész számot olvas be, az első az állomások N száma, a második a centrum C sorszáma, a harmadik pedig a különálló K állomás sorszáma. Tehát minden `kerdes(x,y)` 0-t ad, ha $x = K$ és $y \neq C$ vagy $y = K$ és $x \neq C$, egyébként 1-et.



14. ábra. Egy helyes bemenet ábrája

Pontozás

Ha a `megoldas(m)` eljárással közölt m érték helyes és legfeljebb $4N$ `kerdes` műveletet hajtott végre a program, akkor 2 pont/teszteset jár, ha legfeljebb $2N$ `kerdes` műveletet hajtott végre a program, akkor teljes pontszám jár. A tesztek 40%-ában $N \leq 400$.

Korlátok

$1 < N \leq 10\,000$
Időlimit: 0.1 másodperc
Memórialimit: 32 MB.

Megoldás

Legyen c a centrum, k pedig egy olyan állomás sorszáma, amely csak a centrummal van kapcsolatban. Tekintsük a metró-hálózat M szomszédsági mátrixát, ahol $M[i][j] = 1$, ha az i. és a j. állomás között van metrójárat, egyébként 0. A mátrix szimmetrikus.
A mátrix c. sorában minden tag értéke 1 (kivéve $M[c][c] = 0$),

és a k. oszlopban minden tag értéke 0, kivéve az $M[c][k] = M[k][c] = 1$.

	1	2	3	4	5	6	7	8	9
1	0			0			1		
2		0		0			1		
3			0	0			1		
4	0	0	0	0	0	0	1	0	0
5				0	0		1		
6				0		0	1		
7	1	1	1	1	1	1	0	1	1
8				0			1	0	
9				0			1		0

15. ábra

Stratégia: induljunk a mátrix $ci = 1$ és $cj = 2$ elemétől. Ha a `kerdes(ci,cj)` értéke 0, akkor lépjünk lefelé: $ci++$, egyébként lépjünk jobbra: $cj++$. Ekkor szükségképpen a c. sorban lépünk ki a mátrixból.
Azonban a $c = 1$ extremális eset, ezért ha $kerdes\,(1,\,2) = 0$, akkor a $ci = 2,\ cj = 1$ legyen az indulási hely.
Így legfeljebb $2N - 1$ kérdéssel kitalálható a centrum.

```
1   #include <bits/stdc++.h>
2   #include "tudakozo.h"
3
4   using namespace std;
5   int main(){
6     int N,ci,cj,van;
7     N=hany();
8     van=kerdes(1,2);
9     if (van==1) {
10      ci=1; cj=2;
11    } else {
12      ci=2; cj=1;
13    }
```

```
14        while (ci<=N && cj<=N){
15          if (ci==cj){
16            if (van==1)
17              cj++;
18            else
19              ci++;
20            continue;
21          }
22          van=kerdes(ci,cj);
23          if (van==1)
24            cj++;
25          else
26            ci++;
27        }
28        megoldas(ci);
29
30        return 0;
31      }
```

Feladat: Kutyák és macskák

Egy hosszú, egyenes utcában N kutya és M macska lakik. A kutyákat az $1, 2 \ldots, N$, a macskákat az $1, 2, \ldots, M$ számokkal azonosítjuk (nem feltétlen a lakóhelyeik sorrendjében). Az egyes állatok lakóhelyei megfeleltethetők a számegyenes egy-egy egész értékű pontjának. Hanna most költözött az utcába és nem ismeri, hogy melyik állat hol lakik, de a következőket tudja:

- Semelyik két állat sem lakik ugyanabban a pontban.
- A legkisebb és legnagyobb értékű pontban egyaránt kutya lakik.
- Semelyik két macska sem lakik azonos távolságra valamelyik kutyától.

Hanna szeretné listázni a macskákat a lakóhelyeik sorrendjében. Ehhez kérdéseket tehet fel János bácsinak, aki ismeri az állatok lakóhelyeit. A kérdések a következő alakúak: „Az x vagy az y azonosítószámú macska lakik közelebb a k azonosítójú kutyához?" János bácsi egy ilyen kérdésre x-el vagy y-al felel.
Írj programot, ami segít Hannának minél kevesebb kérdéssel meghatározni a macskák lakóhely szerinti sorrendjét! A növekvő vagy csökkenő sorrendű válasz egyaránt megfelelő.

Könyvtár

A következő eljárást kell megvalósítanod:
`std::vector find_order(int N, int M)` • Az értékelő hívja meg egy új sorrend meghatározásához. Itt kell az `ask` függvény segítségével kérdéseket feltenned és kitalálnod a macskák sorrendjét.

- N a kutyák száma, M a macskák száma az utcában.
- A visszaadott tömb a macskák azonosítószámait tartalmazza, lakóhely szerinti növekvő vagy csökkenő sorrendben!
- Az eljárást egy futtatás során legfeljebb T-szer hívja az értékelő.

A fenti eljárásban a következő függvényt hívhatod:
`int ask(int dog, int cat1, int cat2)`
• $1 \leq dog \leq N$ egy kutya azonosítója. • $1 \leq cat1, cat2 \leq M$ egy-egy macska azonosítói, ahol $cat1 \neq cat2$ • $cat1$ vagy $cat2$ közül azt adja vissza, amelyik közelebb lakik a dog azonosítójú kutyához. • Egy `find_order` hívás során legfeljebb 100 000 kérdést lehet feltenni.
A megoldásodnak tartalmaznia kell a `#include "KutyakMacskak.h"` importáló sort! Ha a megoldásod a standard bemenetről olvas, a standard kimenetre ír, megszakítja a program végrehajtását, main függvényt tartalmaz, vagy más módon eltér a specifikációban foglaltaktól, az rendszerint Interakciós hiba visszajelzést, időlimit túllépést, vagy fordítási hibát eredményez az értékelő rendszerben. Az `ask` függvény nem megengedett paraméterekkel való hívása Interakciós hiba visszajelzést eredményez, csakúgy, mint a megengedett kérdésszám átlépése. Ha a `find_order` függvény által visszaadott tömb helytelen, az hibás válasznak minősül, akkor is, ha a tömb mérete helytelen vagy a tömb nem érvényes azonosítókat tartalmaz.

Gyakorlás

Az értékelőrendszerből letölthető a KutyakMacskak.cpp állomány és egy tesztelésre alkalmas stub program. A projekt mappába másold be, majd add hozzá a projekthez a fejlesztői környezetedben a KutyakMacskak.h, KutyakMacskak.cpp és a stub.cpp állományokat. A megvalósítandó eljárásokat teljes egészében a KutyakMacskak.cpp állományban implementáld! A programod által használt saját függvényeket, egyéb programelemeket is ennek kell tartalmaznia! A szükséges standard könyvtárakat is ebben importáld és a használni kívánt direktívákat is itt add meg (pl. using namespace std;)! Az értékelő rendszerben csak a `KutyakMacskak.cpp` állományt kell beadnod! Ez a tesztelő program a futása során a standard bemenetről olvassa be az adatokat. Elsőként a `find_order` hívások T számát kell megadni. Ezt T teszteset leírása követi. Minden teszteset első sora a kutyák N és a macskák M számát tartalmazza. A második sor N darab 64-bites egész számot tartalmazzon, a kutyák pozícióit a

számegyenesen az azonosítóik sorrendjében. A harmadik sor M darab 64-bites egész számot tartalmazzon, a macskák pozícióit a számegyenesen az azonosítóik sorrendjében. Ezt követően a tesztelő meghívja a `find_order` eljárást az N, M paraméterekkel. Ha az eljárás a futása során helytelen paraméterekkel hívja meg az `ask` függvényt, akkor a tesztelő Interakciós hiba: érvénytelen kérdés kimenettel terminál. A tesztelő program szintén terminál, ha a programod átlépi a megengedett legnagyobb kérdésszámot, ekkor a kimenet Interakciós hiba: túl sok kérdés. A tesztelő program minden tesztesetre két sort ír a standard kimenetre: az első sorba a feltett kérdések Q számát, a második sorba a `find_order` által visszaadott tömb elemeit. A tesztelő program a megoldásod helyességét nem ellenőrzi! Példa Bemenet Kimenet

Példa

Bemenet	Kimenet
2 2 2 3 0 10 9 1 8 5 5 -100 1 51 52 150 0 50 -75 -76 -77	2 2 3 1 11 2 1 5 4 3

A példában az első tesztesetben 2 kutya van a 0 és a 10 pontokban, a macskák rendre a 9,1,8 pontokban vannak. Tekintsük az `ask(1,2,3)` hívást. Az értékelő válasza erre a hívásra 2, mivel a 2-es macska közelebb van az 1-es kutyához, mint a 3-as macska. Hasonlóan az `ask(2,1,3)` hívásra a válasz 1 lenne. Ezt a két kérdést követően már látható, hogy a macskák sorrendje csak 2,3,1 vagy 1,3,2 lehet, így az eljárásnak ezek egyikét kell visszaadnia.

Korlátok

$1 \leq T \leq 10$ $2 \leq N \leq 1000$ $3 \leq M \leq 1000$

Időlimit: 3.0 s
Memórialimit: 64 MB

Részfeladatok

a minta, 0 pont
$N = 2$, 6 pont
$M = 3$, 7 pont
$N, M \leq 20$. 10 pont
$N, M \leq 80$, 12 pont
nincsenek további megkötések 65

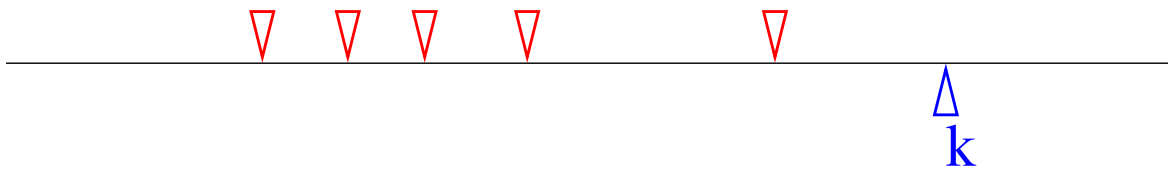
Az értékelő nem adaptív, vagyis a kutyák és a macskák lakóhelyei minden tesztesetben rögzítve vannak a `find_order` eljárás hívása előtt. Az utolsó részfeladatban a pontokat a feltett kérdések száma alapján lehet megszerezni. Ha a megoldásod megsérti a Könyvtár részben leírtakat, vagy legalább egy alkalommal rossz választ ad, akkor 0 pontot kap. Egyébként legyen Q a legtöbb kérdés, amit a beküldésed valamely `find_order` hívás során feltett. Ekkor a

megoldásod pontszáma erre a részfeladatra:

- ha $Q \geq 30000$, akkor 20 pont,
- ha $12000 < Q < 30000$, akkor $20 + 45\,(1 - \sqrt{\frac{Q-12000}{18000}})$ pont,
- ha $Q \leq 12000$, akkor 65 pont.

Megoldás

Rendezésre visszavezető stratégia



16. ábra

Tegyük fel, hogy tudjuk, hogy van olyan k kutya, hogy a macskák M halmazának mindegyike balra (jobbra) van a k kutyától. Ekkor a $ask\,(k, x, y) = x$ rendezési relációval rendezve a macskák helyes sorrendjét kapjuk.

Megjegyzendő, hogy most csak a rendezési algoritmusban alkalmazott összehasonlítások száma fontos! Az egyes (beépített) algoritmusok valamelyest eltérő pontszámot eredményeznek:

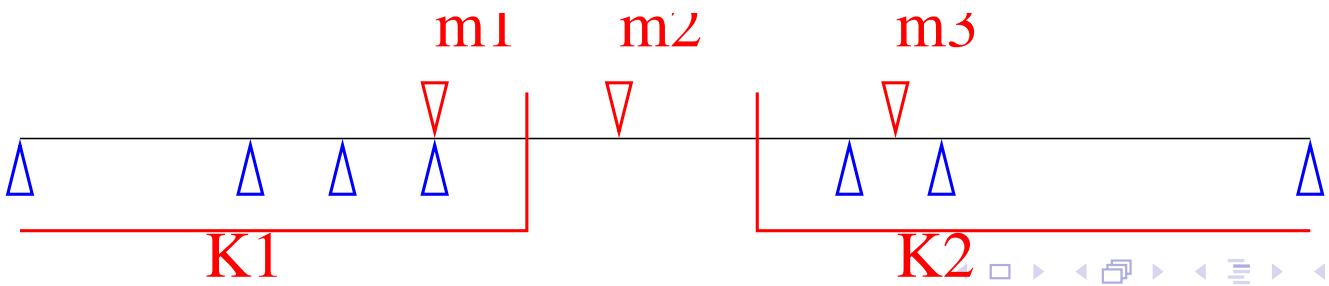
```
sort 74
make_heap(); sort_heap() 77
stable_sort 80
```

Felosztás két részre, majd rendezés

Határozzuk meg három macska sorrendjét: $m1, m2, m3$

$$K1 = \{k : ask(k, m1, m2) = m1\}$$

$$K2 = \{k : ask(k, m2, m3) = m3\}$$



17. ábra

Legyen

$$M1 = \{ M : m \text{ balra van m2-től} \}$$

$$M2 = \{ M : m \text{ jobbra van m2-től} \}$$

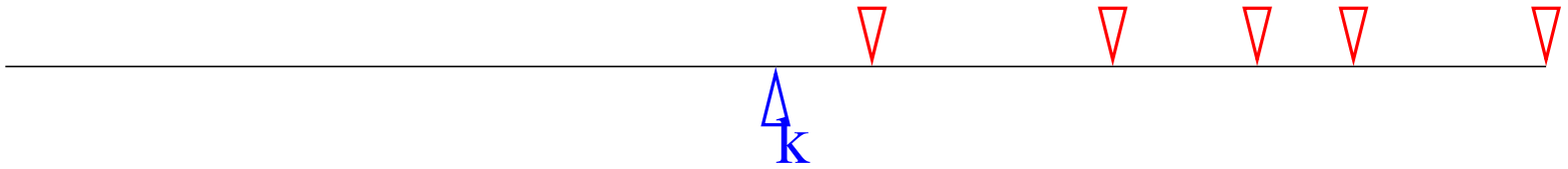
Bármely $k2 \in K2$ a $ask(k2, x, y) = x$ reláció lineáris (teljes) rendezési reláció, ezen reláció szerinti rendezés megfelelő sorrendjét adja az $M1$ halmaznak.
Bármely $k1 \in K1$ a $ask(k1, x, y) = x$ reláció lineáris (teljes) rendezési reláció, ezen reláció szerinti rendezés megfelelő sorrendjét adja az $M2$ halmaznak.

$M1$ és $M2$ előállítása?

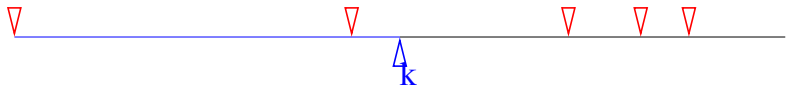
$m \in M1$ akkor és csak akkor, ha van olyan $k1 \in K1$ hogy $ask\,(k1,\ m,\ m2) = m$

$m \in M2$ akkor és csak akkor, ha van olyan $k2 \in K2$ hogy $ask\,(k2,\ m,\ m2) = m$

Hatékony stratégia



18. ábra. $ask\,(k,\ x,\ y)$ rendezési reláció szerint sorrend.



19. ábra. A tényleges szerint sorrend.

Algoritmus

1. lépés: Rendezés $ask(1, x, y) = x$ szerint.

2. lépés: $Jobbra[\,]$ kiszámítása.

3. lépés:

```
1   deque<int> Sorrend({Macskak[0]});
2   for (int j=1; j<M; ++j)
3   if (Jobbra[j])
4     Sorrend.push_back(Macskak[j]);
5   else
6     Sorrend.push_front(Macskak[j]);
```

1. lépés: $ask(1, x, y)$ -szerinti rendezés beszúró rendezéssel, bináris kereséssel.

A kérdések száma $O(M \log(M))$

2. lépés: $Jobbra[\,]$ kiszámítása:

Keressünk olyan k kutyát, hogy

$ask(k, Macskak[0], Macskak[M-1]) = Macskak[M-1]$!

Ekkor az m macska akkor és csak akkor van jobbra, ha

$ask(k, m, Macskak[0]) = m$.

A kérdések száma $O(N+M)$.

Feladatok

1. feladat: rejtett permutáció

`https://cses.fi/problemset/task/3139`

Adott az $1, \ldots, n$ számoknak egy $a_1, \ldots a_n$ rejtett permutációja, ezt kell kitalálnod.

Kommunikáció

Kérdéseket tehetsz fel, amit a standard kimenetre kell kiírni, a válasz a standard bemenetről kell beolvasni.

A programod először olvassa be az n számot. Az alábbi formában tehetsz fel kérdést:

```
? i j
```

A válasz a `YES` szöveg, ha $a_i < a_j$, egyébként pedig a `NO` szöveg.

A kitalált permutációt így kell kiírni:

```
! a1 ... an
```

Korlátok

$1 \leq n \leq 1000$

Legfeljebb 10^4 kérdést tehetsz fel.

Időlimit: 1.0 mp

Memórialimit: 512 MB

Példa

```
3
? 3 2
NO
? 3 1
YES
! 3 1 2
```

2. feladat: csillag molekula központja

Kutatók egy speciális molekulát vizsgálnak. Tudják, hogy a molekula N különböző atomot tartalmaz, amelyek csillag szerkezetet alkotnak. Ez azt jelenti, hogy pontosan egy olyan atom van a molekulában, amelyhez legalább három atom kapcsolódik közvetlenül, és a többi atomhoz legfeljebb kettő. Természetesen a molekula összefüggő, azaz bármely két atomja között van közvetlen szomszédokon keresztül vezető útvonal. A kutatók a molekulát egy speciális mérőműszerrel vizsgálják. A műszer bármely két adott atomjára megad egy közbülső atomot. Írj programot, amely meghatározza a molekula központi atomját, tehát azt, amelyhez legalább három atom kapcsolódik közvetlenül! A programod nem írhat, és nem olvashat semmilyen fájlt, beleértve a standard inputot és outputot is!

Könyvtári műveletek

A mérőműszer használatát a `muszer` könyvtár három művelete biztosítja: atomszam, egyszer kell hívni a program elején, az atomok ($7 \leq N \leq 1000$) számát adja. Az atomokat az 1,...,N számokkal azonosítjuk. kozte, két atom azonosítóját kell argumentumként megadni `kozte(x,y)`; a visszaadott érték egy olyan atom azonosítója, amely a molekulában az x és y atom között van. Ha x és y nem közvetlen szomszédok a molekulában, akkor a visszaadott érték biztosan nem x és nem y. Ha x és y közvetlen szomszédok, akkor a visszaadott érték vagy x, vagy y. központ, a program végén kell hívni, a kiszámított eredmény közléséhez! Az argumentum a központi atom sorszáma legyen!

Könyvtár

```
#include "muszer.h"
int atomszam(void);
int kozte(int x, int y);
void kozpont(int x);
```

Letölthető `csillag.zip` állomány tartalmazza `muszer.h` az értékelő `muszer.o` modul C++ le-fordított program Win és Linux verzióját és egy minta `megoldas.cpp` programot. A `muszer` a standard bemenetről egy egész számot olvas be, az atomok N számát. Az értékelő a standard kimenetre három sort ír. Az első sor a pontszám, a második a feltett kérdések száma, a harmadik pedig az értékelési üzenet.

Parancssori fordítás

```
g++ -std=gnu++20 -O2 -static -o megoldas
megoldas.cpp muszer.o
```

Fejlesztőkörnyezetben

A projekthez add hozzá a `muszer.o`, a `muszer.h` és a `megoldas.cpp` fájlokat.

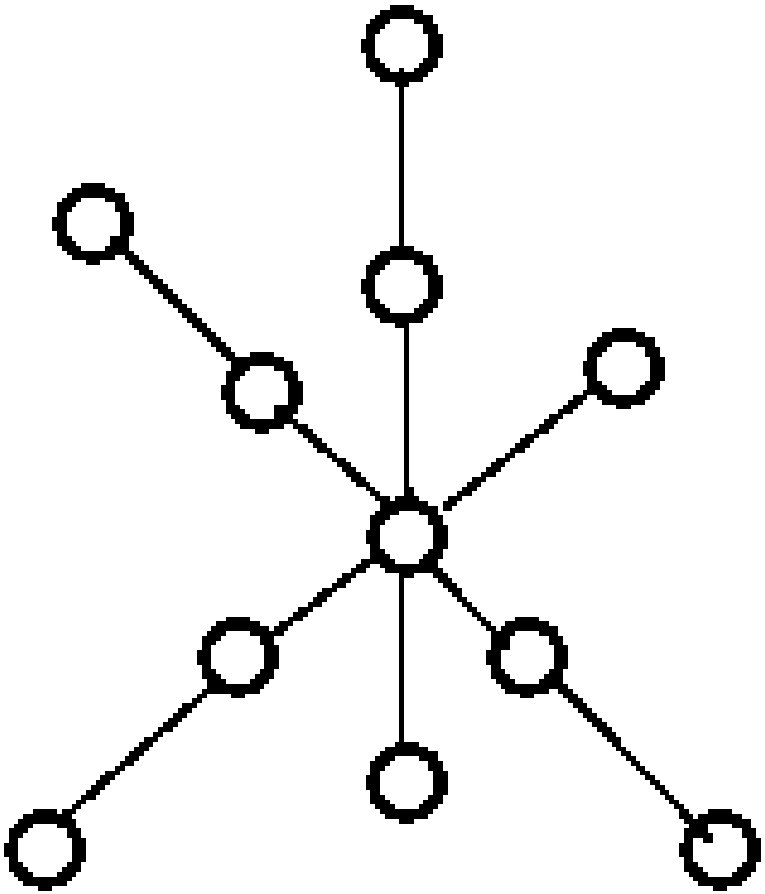
Korlátok

$1 \leq n \leq 1000$ Időlimit: 1.0 mp.

Memórialimit: 32MB

Pontozás

Ha a Központ eljárással közölt atom helyes és legfeljebb $2 * N + 10$ `kozte` műveletet hajtott végre a program, akkor teljes pontszám jár. Ha legfeljebb $3 * N$ `kozte` műveletet hajtott végre a program, akkor 50% pont jár. A tesztek 30%-ában $N \leq 100$.
Az értékelő adaptív.



3. feladat: Találka

A Négyszögletű Kerekerdőben N barát él, ahol N **páratlan** pozitív egész. A barátokat 1-től N-ig sorszámozzuk. A barátok mindegyike a saját házában él. Az N házat $N-1$ darab kétirányú útszakasz köti össze úgy, hogy bármely két ház között pontosan egy útvonal vezet.

A barátok szeretnének összegyűlni az egyikük házában társasjátékozni úgy, hogy ehhez összességében a lehető legkevesebbet kelljen utazniuk. Formálisan, az i-edik barát és a j-edik barát házának $dist(i, j)$ távolsága a két ház közötti útszakaszok darabszáma. Azt az O házat keresik, amelyre teljesül, hogy $\sum_{i=1}^{N} dist(i, O)$ a lehető legkisebb.

A barátok képtelenek meghatározni az optimális O házat ennyi személyre, ezért a segítségedet kérik. Sajnos nem rendelkezel az erdő térképével és a barátok sem tudják megmondani, hogy pontosan melyik ház melyik útszakaszokkal van összekötve. Azt viszont ki tudják számolni, hogy ha csak egyszerre **három különböző** A, B és C barátnak kellene találkoznia, akkor az N ház közül melyik lenne az optimális – azaz melyik X ház esetén lenne a legkisebb $dist(A, X) + dist(B, X) + dist(C, X)$ összeg.
Írj programot, ami meghatározza az N barát találkahelyét úgy, hogy legfeljebb 500 000 alkalommal megkérdezi három barátról, hogy melyik ház lenne az ő optimális találkahelyük. Belátható, hogy a megadott feltételek mellett minden szabályos kérdésre egyértelmű válasz adható, és az optimális találkahely is egyértelmű.

Könyvtár

A következő eljárást kell megvalósítanod:

```
int optimal_place(int N)
```

Az értékelő hívja meg a program futásának kezdetén. Itt kell az `organize_meeting` függvény hívásával információkat szerezned és kitalálnod az optimális találkahelyet.

N a barátok száma (és egyben a házak száma is).

A visszaadott érték az optimális találkahely O sorszáma.

A fenti eljárásban a következő függvényt hívhatod:

```
int organize_meeting(int A, int B, int C)
```

$1 \leq A, B, C \leq N$ három **különböző** barát sorszámai.

A függvény visszaadja az A, B és C barátok optimális találkahelyének X sorszámát.

Egy `optimal_place` hívás során legfeljebb 500 000-szer hívható meg.

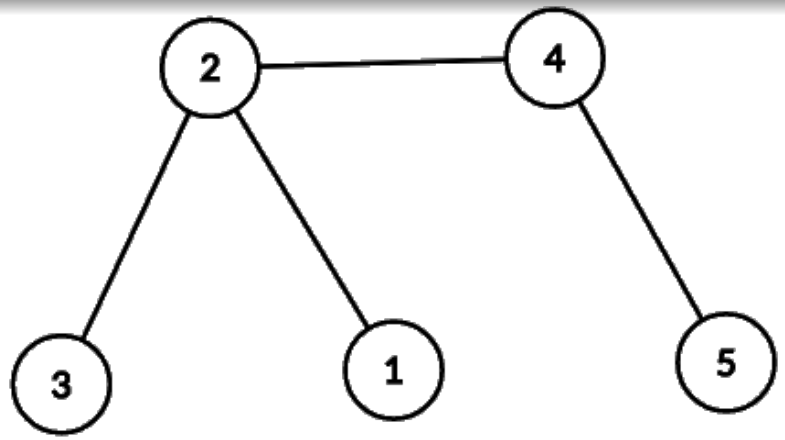
A megoldásodnak tartalmaznia kell a `#include "talalka.h"` importáló sort.
Ha a megoldásod a `standard bemenet`ről olvas, a `standard kimenet`re ír, megszakítja a program végrehajtását, `main` függvényt tartalmaz, vagy más módon eltér a specifikációban foglaltaktól, az rendszerint `Protocol Violation` visszajelzést, időlimit túllépést, vagy fordítási hibát eredményez az értékelő rendszerben.
Az `organize_meeting` függvény nem megengedett paraméterrel való hívása `Output isn't correct` visszajelzést eredményez, csakúgy, mint a megengedett kérdésszám átlépése. Az értékelő ezekben az esetekben a hiba okát is megjeleníti (`Too many calls` vagy `Invalid argument`).
Ha az `optimal_place` függvény által visszaadott érték helytelen, az hibás válasznak minősül, akkor is, ha az érték nem egy létező ház sorszáma.
Az értékelő **nem adaptív**, azaz az útszakaszok elhelyezkedése rögzítésre kerül az `optimal_place` eljárás hívása előtt.
Az értékelőrendszerből letölthető a `talalka.cpp` állomány és egy

tesztelésre alkalmas `stub` program. A projekt mappába másold be, majd add hozzá a projekthez a fejlesztői környezetedben a `talalka.h`, `talalka.cpp` és a `stub.cpp` állományokat.
A megvalósítandó eljárásokat teljes egészében a `talalka.cpp` állományban implementáld. A programod által használt saját függvényeket, egyéb programelemeket is ennek kell tartalmaznia. A szükséges standard könyvtárakat is ebben importáld és a használni kívánt direktívákat is itt add meg (pl. `using namespace std;`). Az értékelő rendszerben csak a `talalka.cpp` állományt kell beadnod.
Ez a tesztelő program a futása során a `standard bemenetről` olvassa be az adatokat. Az első sorban a házak N számát kell megadni. A következő $N-1$ sor mindegyike két pozitív egész számot tartalmazzon: két olyan ház sorszámát, amelyek között van egy útszakasz.
Ezt követően a tesztelő meghívja az `optimal_place` eljárást az N paraméterrel. Ha az eljárás a futása során helytelen paraméterekkel hívja meg az `organize_meeting` függvényt, akkor a tesztelő `Invalid argument` kimenettel terminál. A

tesztelő program szintén terminál, ha a programod átlépi a megengedett legnagyobb kérdésszámot, ekkor a kimenet `Too many calls`.
A tesztelő program végül két sort ír a `standard kimenetre`: az első sorba az `optimal_place` által visszaadott O sorszámot, a második sorba a felhasznált kérdések számát. A tesztelő program a megoldásod helyességét nem ellenőrzi.

Példa

Bemenet	Kimenet
5 1 2 2 3 2 4 4 5	2 3



A példa gráfja

A példában a tesztelő program egy lehetséges bemenete és egy helyes kimenete látható 3 feltett kérdéssel. Példák lehetséges kérdésekre és a rájuk kapott válaszokra:

`organize_meeting(1, 2, 3)` esetén $X = 2$.

`organize_meeting(4, 2, 5)` esetén $X = 4$.

`organize_meeting(1, 5, 3)` esetén $X = 2$.

Korlátok

$3 \leq N \leq 25\,000$ és N páratlan

Pontozás

0 pont: minta

9 pont: $N \leq 100$, minden házat legfeljebb két másik házzal köt össze útszakasz

8 pont: $N \leq 1000$, minden házat legfeljebb két másik házzal köt össze útszakasz

14 pont: minden házat legfeljebb két másik házzal köt össze útszakasz

19 pont $N \leq 100$

20 pont: $N \leq 1000$

30 pont: nincsenek további megkötések